KKT

Real Estate ERP · Marketing Site · Customer Portal

Technical & Functional Specification

Version 12.1 · 25 October 2026

Derived from the production code base (FastAPI · React · MongoDB · Expo)

Audience: owners, sales leadership, IT & integration teams

Confidential — prepared for prospective enterprise customers

Contents

Part I — Product & Architecture 7

1. Executive summary . . . 7
   - 1.1 The three surfaces on one backend . . . 7
   - 1.2 Personas & roles . . . 7
2. System architecture . . . 7
   - 2.1 Stack . . . 7
   - 2.2 Folder structure . . . 8
   - 2.3 Request lifecycle & middleware chain . . . 8
   - 2.4 Host-aware mounting & the three-deployment recipe . . . 9
   - 2.5 Caching layers . . . 9
   - 2.6 Background jobs & startup sequence . . . 9
3. Security & privacy . . . 10
   - 3.1 Authentication . . . 10
   - 3.2 Authorisation & scoping . . . 10
   - 3.3 Data protection . . . 10
   - 3.4 Permission registry (generated from permissions.py) . . . 11
   - 3.5 Role × permission matrix (6 system roles × 36 permissions) . . . 12

Part II — Data model 14

4. Data model — collections (generated from the live database) . . . 14
   - 4.1 Core entity relationships . . . 46
   - 4.2 State machines . . . 47

Part III — Modules 49

5. Master data & selection-driven design . . . 49
6. Flat generation algorithm (services/flat_generation.py) . . . 49
7. Pricing engine . . . 49
8. Projects, blocks, flats & inventory UI . . . 50
9. Customers & KYC (routers/customers.py, services/kyc.py) . . . 50
10. Leads pipeline (routers/leads.py, routers/visits.py) . . . 50
11. Booking engine (routers/bookings.py, pages/BookingWizard.jsx) . . . 51
    - 11.9 Edge cases — booking engine . . . 51
12. Construction stages & tower status (routers/construction.py) . . . 52
13. Payment plans & the auto-dues engine (routers/bookings.py plans, services/dues_engine.py) 52
    - 13.7 Edge cases — plans & auto-dues . . . 53
14. Payment ledger engine (routers/payments.py, services/ledger.py, services/money.py) . . . 54
    - 14.1 Worked example — ₹1 Cr agreement, ₹60 L cash + ₹40 L account, Standard CLP (seeded BK-2526-0001) . . . 54

14.9 Edge cases — payment ledger . . . 56
15. Documents & media (routers/documents.py, services/thumbs.py) . . . 56
16. Service requests (routers/support.py) . . . 57
17. Notifications (routers/notifications.py, services/portal_notify.py, events.py) . . . 57
17.1 Event catalogue . . . 57
17.2 Targeting and delivery . . . 57
17.3 Edge cases . . . 58
18. Reports & PDFs (routers/reports.py, routers/pdfs.py, services/pdf.py) . . . 58
18.1 Report contract . . . 58
18.2 Document PDFs (routers/pdfs.py) . . . 59
19. Website + CMS (routers/cms.py, routers/public.py, /app/site, pages/cms/*) . . . 59
19.1 Pages and toggles . . . 59
19.2 Public API hygiene . . . 60
19.3 Enquiry flow . . . 60
19.4 Brochure lead gate . . . 60
19.5 Site-visit availability, booking, ICS, cancel . . . 60
20. Customer portal (Expo, /app/mobile) . . . 61
20.1 Architecture . . . 61
20.2 Auth isolation . . . 61
20.3 Screens . . . 61
20.4 Offline cache, notifications, responsive shell . . . 61

Part IV — Quality, operations, roadmap 63

21. Testing & verification . . . 63
21.1 Test suites (generated from /app/backend/tests) . . . 63
21.2 Independent audits & before/after metrics . . . 63
22. Operations . . . 64
22.1 Environment variables . . . 64
23. Known limitations & roadmap . . . 65

Part V — ERP v2 (Phases 10–12): packaging, canonical backend, design system, new modules 66

24. Single-domain deployment package (frontend/scripts/package-build.js, package-assets.js, dev.js) . . . 66
24.1 Goal and layout . . . 66
24.2 Router-loader . . . 66
24.3 Same-origin API resolution . . . 67
24.4 Preview = production . . . 67
24.5 Deploy checklist (owner) . . . 67
25. Canonical backend hardening (Phase 11A) . . . 67
25.1 Catalogues — selection-driven data entry (routers/catalogues.py, catalogue_items) . . . 67

25.2 Project setup (routers/project_setup.py) . . . 68
25.3 CRM: communications, timeline, three ledgers (routers/crm.py, services/ledgers.py) . . . 68
25.4 Read models, bootstrap and caching . . . 68
26. Design system "Royal Black & Warm White" (frontend/src/components/v2/) . . . 68
27. v2 screens and their algorithms . . . 69
27.1 Home (pages/v2/Home.jsx) . . . 69
27.2 Projects · Workspace · Setup wizard (pages/v2/Projects.jsx, ProjectWorkspace.jsx, ProjectSetup.jsx) 69
27.3 Customers list · form · Customer 360 (pages/v2/Customers.jsx, CustomerForm.jsx, Customer360.jsx)
27.4 Leads & Visits (pages/v2/Leads.jsx) . . . 69
27.5 Bookings · wizard · detail (pages/v2/Bookings.jsx, BookingWizard.jsx, BookingDetail.jsx) . . . 70
27.6 Payments · Post Payment · Roznamcha (pages/v2/Payments.jsx, components/v2/PostPayment.jsx, pages/Roznamcha.jsx) . . . 70
27.7 Service (pages/ServiceRequests.jsx) . . . 70
27.8 Reports v2 (pages/Reports.jsx, routers/reports.py) . . . 70
27.9 Settings → Catalogues, structured masters, Users & Roles (pages/v2/Catalogues.jsx, pages/master/*, pages/Users.jsx, pages/Roles.jsx) . . . 70
28. Phase 12 polish & measurements . . . 70

Appendices **72**

A. API map (generated from the OpenAPI specification) . . . 72
admin . . . 72
audit . . . 72
auth . . . 72
blocks . . . 72
bookings . . . 73
bootstrap . . . 73
catalogues . . . 73
cms . . . 74
construction . . . 74
crm . . . 74
customers . . . 75
dashboard . . . 75
documents . . . 75
flats . . . 75
home . . . 75
imports . . . 76
leads . . . 76
master . . . 76
milestones (legacy alias) . . . 79
notifications . . . 79
parking . . . 79

payments . . . 79
pdf . . . 80
portal . . . 80
portal-admin . . . 81
project-setup . . . 81
projects . . . 81
public . . . 82
public (visits & brochure) . . . 82
rate-cards . . . 82
reports . . . 83
roles . . . 83
roznamcha . . . 83
search . . . 83
settings . . . 84
site visits . . . 84
support . . . 84
system . . . 85
users . . . 85
B. Error code catalogue (generated) . . . 85
C. Settings keys . . . 91
D. Status & enum tables (generated) . . . 91
E. Seeded demo data & credentials . . . 91
F. Glossary . . . 91

Part I — Product & Architecture

1. Executive summary

KKT is a single-backend real-estate operating system for Indian developers. One FastAPI service (`/app/backend`) powers three front doors: the **ERP** (`/app/frontend`, React, served under `/erp`), the public **marketing website** (`/app/site`, React, served at `/`) and the **customer portal** (`/app/mobile`, Expo — web, iOS and Android). Everything a developer does between launching a tower and handing over keys is modelled: master data, floor-plan driven inventory generation, pricing, CRM, construction-linked payment plans with automatic demands, an integer-paise payment ledger with receipts and Roznamcha, documents and photos, service requests, notifications, reports, a content-managed website with lead capture (enquiries, brochure lead gate, site-visit scheduler) and a customer portal that mirrors the ledger.

> *Design principles —* **selection, not typing** *(every repeated value is a master);* **track, don't gate** *(the ledger never refuses a legitimate rupee, it reports it);* **one truth** *(demands, payments and the ledger are derived, idempotent and re-buildable);* **auditability** *(every write publishes an event that lands in an immutable audit log);* **additive public contracts** *(portal and site APIs only grow).*

1.1 The three surfaces on one backend

Surface	Path / host	Audience	Auth
ERP	`/erp` on the main domain (`frontend/scripts/dev.js` serves the production build)	Owners, project admins, sales, accounts, marketing, site engineers	JWT `aud=erp`, 15-min access + rotating refresh, optional TOTP MFA
Marketing site	`/` on the main domain (same Node server, separate React build)	Prospective buyers	Public; rate-limited, honeypot-protected forms
Customer portal	Expo app; `/portal` redirects to the configured Expo host	Home buyers	JWT `aud=portal`, isolated token family, forced first-login password change

1.2 Personas & roles

Persona	System role	What they do
Owner / MD	Super Admin	Everything; sees integrity reports, audit log, documentation, reseed
Project head	Project Admin	Structure, inventory, pricing, bookings, construction, leads & visits for assigned projects
Sales executive	Sales	Leads, site visits, bookings, customers for assigned projects
Accounts	Accounts	Payments, receipts, reversals, Roznamcha, finance reports (all projects)
Marketing	Marketing	CMS, blog, gallery, enquiries, lead read, visits read
Site engineer	Site Engineer	Construction stages, photos (customer-hidden photos allowed)
Employee	Employee	Read-only dashboards and lists; no finance access
Buyer	(portal customer)	Bookings, schedule, ledger, receipts, documents, construction photos, tickets, notifications

The full permission registry and the role × permission matrix are generated from `permissions.py` in §3.4–3.5.

2. System architecture

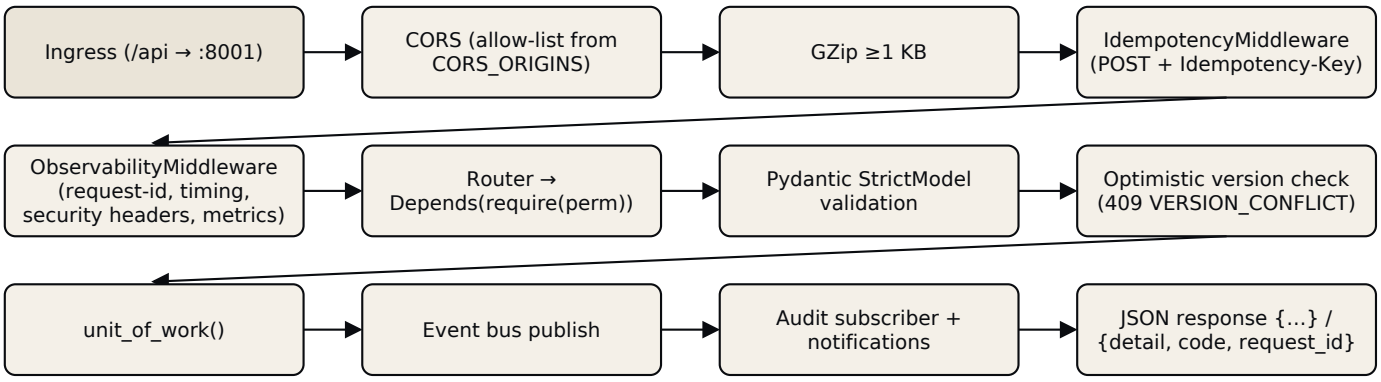
2.1 Stack

Layer	Technology	Notes
API	FastAPI 0.11x, Python 3.11, Motor (async MongoDB), Pydantic v2 `StrictModel` (`extra="forbid"`)	Supervisor-managed on `0.0.0.0:8001`; every route under `/api`
Database	MongoDB (standalone in preview; replica set recommended)	`MONGO_URL`, `DB_NAME` from env only; UUID string `id`, `_id` never leaves the API
ERP	React 18 (CRA), TanStack Query, shadcn/ui, Tailwind, Recharts, lucide	Design tokens (v2 Royal Black & Warm White): ink `#0A0C10`, warm `#FBF9F4`/`#F4F0E8`, gold `#C9A227`, Manrope + serif display — see §26
Site	React 18 (CRA), Tailwind, framer-motion, react-helmet-async, `fetch` client	Cormorant + Manrope, route-level code splitting
Portal	Expo (React Native + web)	Separate repo folder `/app/mobile`; consumes `/api/portal/*` only
Files	Emergent object storage via `storage.py` (`put_object/get_object`)	Thumbnails via Pillow (WebP)
PDFs	reportlab (`services/pdf.py`)	Receipts, demand letters, allotment letters, statements, brochures, this document

2.2 Folder structure

```
/app/backend
  server.py            app factory, middleware chain, lifespan (indexes, migrations, seeds, loops)
  deps.py              JWT, auth dependencies, require(perm), project scoping, rate limiter
  permissions.py       PERMISSION_GROUPS, SYSTEM_ROLES
  utils.py             ApiError, StrictModel, base_doc, paginate, ListParams, audit, csv
  uow.py               unit_of_work(): Mongo transactions when available, compensation otherwise
  events.py            in-process event bus → audit subscriber, roles_version bump
  infra.py             ObservabilityMiddleware (request-id, timing, security headers, metrics), IdempotencyMiddlew
  crypto.py            Fernet field encryption, masking
  storage.py           object storage adapter
  routers/             auth users roles projects blocks flats rate_cards parking customers leads bookings
                       payments roznamcha milestones construction documents support notifications reports
                       pdfs dashboard dashboard_widgets search settings imports audit admin cms public portal visi
  services/            flat_generation pricing(kyc,money,numbering) ledger dues_engine integrity thumbs pdf
                       portal_notify counters ttlcache filesig
  migrate_phase7.py    legacy milestones → payment-plan installments (idempotent)
  seed*.py             deterministic demo data (guarded by settings markers)
  tests/               pytest suites (hit the preview URL)
/app/frontend          ERP (src/pages, src/components/common, src/hooks, scripts/dev.js = prod server)
/app/site              marketing site (src/pages, SiteShell.jsx, LeadGate.jsx, VisitBooking.jsx)
/app/mobile            Expo customer portal (untouched by ERP builds)
/app/docs              this documentation (markdown + generated appendices + PDF)
```

2.3 Request lifecycle & middleware chain



1. **Request id & timing** — `X-Request-Id` is generated (or echoed), the duration is logged as structured JSON and aggregated into `/api/metrics` (`infra.metrics_snapshot`).

2. **Security headers** — `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy`, `Permissions-Policy`, HSTS when behind TLS.
3. **Rate limiting** — `deps.enforce_rate_limit(key, limit, window)` is a Mongo-backed sliding window (`rate_limits` collection) keyed by IP/route: login 10/min, enquiries 5/min, brochure requests 5/min, site visits 5/min, portal login 10/min. Returns 429 `RATE_LIMITED` with `retry_after`.
4. **Authentication** — `Depends(current_user)` decodes the JWT (`aud` must be `erp`; portal tokens carry `aud=portal` and are rejected by ERP routes), loads the user (without secrets) and verifies `is_active`, forced password change and MFA status.
5. **Permission** — `require("perm.key")` checks the role's permission list (cached per process; invalidated by `roles_version` bumps published on role edits) and attaches `project_scope_filter(user)` for list endpoints.
6. **Version check** — writable documents carry `version`; PATCH bodies send the version they saw; a mismatch is a 409 `VERSION_CONFLICT` (`guarded_update`).
7. **Idempotency** — POSTs that create money or inventory require `Idempotency-Key` (`IDEMPOTENCY_KEY_REQUIRED`); the middleware stores the first response per key for 24 h and replays it.
8. **Unit of work** — `uow.unit_of_work()` opens a Mongo transaction when the server supports it (`detect_transaction_support()`); otherwise it records compensation callbacks (`on_rollback`) so multi-document writes roll back on failure.
9. **Event bus → audit** — `events.bus.publish("entity.action", payload)`; `audit_subscriber` writes the immutable `audit_logs` row (actor, entity, before/after diff, ip, request id); other subscribers fan out notifications.

2.4 Host-aware mounting & the three-deployment recipe

`frontend/scripts/dev.js` is both the dev proxy and the production preview server. Since Phase 12 the production artefact is a **single packaged build** (`yarn build` → `build/` with the marketing site at `/`, the ERP under `/erp/` and the Expo web portal under `/portal/`, plus an inline router-loader in the root `index.html`); `dev.js` serves that bundle 1:1 (hashed assets `Cache-Control: immutable`, root-index fallback for every non-`/api` path, Brotli/Gzip). The legacy three-host recipe (`www.`, `erp.`, `portal.`) remains supported — see §22 and §24.

2.5 Caching layers

Layer	Mechanism	Invalidation
Permissions	per-process dict keyed by role id + `roles_version`	role edit publishes `roles_version` bump
Public API	`ETag` + `If-None-Match` → 304, `Cache-Control: public, max-age` on `/api/public/*`; `ttlcache` 60 s for availability	writes call `ttlcache.invalidate(prefix)`
Media	thumb/medium WebP variants pre-generated; `Cache-Control: public, max-age=31536000, immutable`, strong ETag	storage keys are content-addressed per upload
ERP data	TanStack Query `staleTime` 15 s, `useList` reload after mutations, unread-count polling 30 s	mutation hooks
Static assets	hashed filenames, immutable	new build hash

2.6 Background jobs & startup sequence

Job	Schedule	Code
Daily dues	00:30 IST (and once at startup)	`server._dues_loop` → `dues_engine.raise_due_installments` per Active booking

Job	Schedule	Code
Overdue sweep	hourly	`_overdue_loop` marks demands `Overdue`, notifies customer + RM once
Integrity check	startup + nightly	`services.integrity.integrity_report` → log + Super Admin notification when `ok=False`
Notification sweep	startup	`portal_notify.sweep()` normalises legacy types, deletes orphaned/voided references
Media prewarm	startup (background)	`thumbs.prewarm_variants` generates missing thumb/medium variants
Site-visit nudge	every 6 h	`visits.nudge_loop` — past Confirmed/Rescheduled visits → "Mark outcome" to RM

Startup (`server.lifespan`): create indexes (unique `(booking_id, installment_seq)` partial on `voided≠true`, TTL on `brochure_tokens`, text/compound list indexes) → sync system roles (`roles_version`) → migrations (`migrate_phase7`) → backfills (`integrity.backfill_demand_installments`) → seed-if-empty (phases 1–9, each guarded by a `settings` marker) → integrity report log → start loops.

3. Security & privacy

3.1 Authentication

Control	Implementation (,)
Password hashing	bcrypt via passlib; policy ≥ 8 chars with letter + digit; history of last 5 (`PASSWORD_HISTORY`)
Access token	HS256 JWT, `aud` ∈ {`erp`, `portal`}, `exp` = 15 min (`ACCESS_TOKEN_MINUTES`)
Refresh token	opaque, hashed at rest in `refresh_tokens` with a `family` id; 7 days (`REFRESH_TOKEN_DAYS`); each use rotates; reuse of a consumed token revokes the whole family (`REFRESH_TOKEN_REUSED`)
Lockout	5 failed attempts → 15-min lock (`LOCKED`); counters on the user document
Rate limit	10 logins / 60 s / IP (Mongo sliding window)
MFA	TOTP (pyotp) enrol → verify → login returns `mfa_required` + short-lived `mfa_token` (aud `erp`) → `/auth/mfa/login`; disable requires password + code
Sessions	`/auth/sessions` lists refresh families with device/IP; revoke one or all
Portal	separate login, `aud=portal`, customer `must_change_password` on first login, strict serializers (`routers/portal.py`)

3.2 Authorisation & scoping

Permissions are flat strings grouped for the UI (§3.4). `require(perm)` returns the user; list endpoints merge `project_scope_filter(user)` — `{}` for all-project roles, `{"project_id": {"$in": user.project_ids}}` otherwise. Super Admin bypasses with `permissions="*"`. Dangerous settings (`settings.manage`, `audit.view`, reseed, integrity repair, documentation) are Super Admin only.

3.3 Data protection

- **Field encryption** — PAN, Aadhaar and bank account numbers are stored as Fernet ciphertext (`crypto.encrypt_field`, key `FIELD_ENCRYPTION_KEY`) plus an HMAC-SHA256 `*_hash` (`kyc.secure_hash`) for dedupe/equality search; API responses return masked values (`XXXXX1234F`, `XXXX XXXX 1234`) unless the caller holds `customer.kyc_view` and asks `?reveal=true`, which is audited (`customer.kyc_reveal`).
- **Validation** — PAN regex, Aadhaar Verhoeff checksum (`kyc.verhoeff_valid`), IFSC format, Indian mobile normalisation.

- **Uploads** — `services/filesig.py` checks magic bytes against the declared MIME (PDF/JPEG/PNG/WebP), size limits, sensitive flag; files live in object storage, never in Mongo.
- **Signed links** — brochure downloads (15 min), site-visit manage/ICS links, portal document links: HMAC-signed payload (`visits.sign/unsign`), expiry → 410 `LINK_EXPIRED`, tamper → 404.
- **Public allow-list** — `/api/public/*` serializers whitelist fields (`has_brochure`, prices only when `show_prices`); `tests/test_portal_leaks.py` asserts portal payloads never contain internal ids, costs or secrets.
- **Audit log** — append-only `audit_logs {id, at, actor_id, actor_name, action, entity_type, entity_id, before, after, ip, request_id, company_id}`; no update/delete endpoints exist.
- **Erasure & export** — `POST /api/customers/{id}/erase` (blocked while an Active/Completed booking exists → 409 `ACTIVE_BOOKING`) anonymises PII and keeps financial rows; `/api/admin/export` streams a JSON backup.
- **OWASP outcome** — injection (typed queries, no string concatenation), broken auth (above), sensitive data (encryption + masking), XXE n/a, access control (`require` + scoping + tests), misconfiguration (headers, CORS allow-list), XSS (React escaping; CMS HTML sanitised), insecure deserialisation (Pydantic strict), logging (request ids, audit), SSRF (no outbound fetch of user URLs).
- **Known limitations** — single-tenant per deployment; rate limits are per-process-safe but per-IP (behind a shared NAT several users share a bucket); no WebAuthn; refresh tokens are not bound to device fingerprints.

3.4 Permission registry (generated from `permissions.py`)

Group	Permission	Description
Administration	`users.manage`	Manage users
Administration	`roles.manage`	Manage roles & permissions
Administration	`settings.manage`	Manage company settings
Administration	`audit.view`	View audit log
Master Data	`master.read`	View master data
Master Data	`master.write`	Create / edit master data
Master Data	`catalogue.write`	Add / edit dropdown catalogue values (inline '+ Add new')
Projects & Inventory	`project.read`	View projects
Projects & Inventory	`project.write`	Create / edit projects
Projects & Inventory	`block.write`	Create / edit blocks
Projects & Inventory	`flat.write`	Edit flats & pricing
Projects & Inventory	`flat.status`	Change flat status (Hold/Block)
Projects & Inventory	`construction.update`	Update tower construction status (alias: milestone.update)
Projects & Inventory	`construction.undo`	Undo last completed construction stage (alias: milestone.undo)
Customers & Leads	`customer.read`	View customers
Customers & Leads	`customer.write`	Create / edit customers
Customers & Leads	`customer.kyc_view`	View unmasked KYC (PAN/Aadhaar)
Customers & Leads	`lead.read`	View leads
Customers & Leads	`lead.write`	Create / edit leads
Customers & Leads	`visit.read`	View site visits
Customers & Leads	`visit.write`	Create / manage site visits
Bookings	`booking.read`	View bookings

Group	Permission	Description
Bookings	booking.create	Create bookings
Bookings	booking.cancel	Cancel bookings
Bookings	agreement.edit	Edit agreement values
Payments	payment.read	View payments
Payments	payment.post	Post payments
Payments	payment.reverse	Reverse payments
Payments	receipt.print	Print receipts
Reports & Website	report.view	View reports
Reports & Website	cms.read	View website CMS
Reports & Website	cms.write	Edit website CMS
Customer Portal & Service	customer.portal_manage	Manage customer portal access
Customer Portal & Service	service.read	View service requests
Customer Portal & Service	service.write	Reply / update service requests
Customer Portal & Service	service.assign	Assign service requests

3.5 Role × permission matrix (6 system roles × 36 permissions)

Permission	Super Admin	Project Admin	Accounts	Marketing	Sales	Employee
users.manage	●					
roles.manage	●					
settings.manage	●					
audit.view	●		●			
master.read	●	●	●	●	●	●
master.write	●					
catalogue.write	●	●	●		●	
project.read	●	●	●	●	●	●
project.write	●					
block.write	●	●				
flat.write	●	●				
flat.status	●	●				
construction.update	●	●				
construction.undo	●					
customer.read	●	●	●	●	●	
customer.write	●	●			●	
customer.kyc_view	●	●	●			
lead.read	●	●		●	●	
lead.write	●	●		●	●	
visit.read	●	●		●	●	
visit.write	●	●			●	
booking.read	●	●	●		●	●
booking.create	●	●			●	
booking.cancel	●	●				
agreement.edit	●					

Permission	Super Admin	Project Admin	Accounts	Marketing	Sales	Employee
payment.read	●	●	●		●	
payment.post	●		●			
payment.reverse	●		●			
receipt.print	●	●	●			
report.view	●	●	●			
cms.read	●			●		
cms.write	●			●		
customer.portal_manage	●		●		●	
service.read	●	●	●		●	●
service.write	●	●	●		●	
service.assign	●	●	●		●	

Role scope notes:

- **Super Admin** — Full access to everything. Project scope: all projects.
- **Project Admin** — Manages project structure, inventory and sales for assigned projects. Project scope: assigned projects only.
- **Accounts** — Payments, receipts and financial reporting. Project scope: all projects.
- **Marketing** — Manages the public website, blog, gallery and enquiries. Project scope: all projects.
- **Sales** — Customers, leads and bookings. Project scope: assigned projects only.
- **Employee** — Read-only access. Project scope: assigned projects only.

Part II — Data model

4. Data model — collections (generated from the live database)

69 collections. Field types are inferred from a representative document; `id` is always a UUID string (Mongo `_id` is never exposed). Money is stored as integer paise (`*_paise`). Dates are ISO strings.

`amenities` (12 docs)

Field	Type	Example / note
`id`	uuid (str)	"c89c8eac-d08f-4870-8f28-af3b7e7de484"
`company_id`	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
`created_at`	datetime (ISO)	"2026-10-25T08:27:39.538498+00:00"
`updated_at`	datetime (ISO)	"2026-10-25T08:27:39.538498+00:00"
`created_by`	str	"system"
`is_active`	bool	true
`deleted_at`	null	null
`version`	int	1
`name`	str	"Swimming Pool"
`category`	str	"Lifestyle"
`icon`	str	"waves"
`description`	str	"Temperature-controlled pool with deck"

Indexes: `id:1` unique, `company_id:1, is_active:1, name:1`

`area_types` (6 docs)

Field	Type	Example / note
`id`	uuid (str)	"ad8a0235-87ad-43a5-8c2f-dcd67461dd65"
`company_id`	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
`created_at`	datetime (ISO)	"2026-10-25T08:27:39.528907+00:00"
`updated_at`	datetime (ISO)	"2026-10-25T08:27:39.528907+00:00"
`created_by`	str	"system"
`is_active`	bool	true
`deleted_at`	null	null
`version`	int	1
`name`	str	"Carpet Area"
`code`	str	"CARPET"
`sort_order`	int	1

Indexes: `id:1` unique, `company_id:1, is_active:1, name:1`

`audit_logs` (9748 docs)

Immutable audit trail (actor, action, entity, before/after, ip, request id).

Field	Type	Example / note
`id`	uuid (str)	"65fa11ff-c33f-4e02-9765-ac8d53031309"
`company_id`	null	null

Field	Type	Example / note
actor_id	str	"system"
actor_name	str	"System"
action	str	"company.create"
entity_type	str	"company"
entity_id	uuid (str)	"4b16375d-9813-4d23-8a4d-f62a253528a5"
before	null	null
after	object	16 keys
ip	str	""
created_at	datetime (ISO)	"2026-10-23T22:19:35.689982+00:00"

Indexes: `entity_type:1, entity_id:1`, `created_at:-1`, `actor_id:1`, `company_id:1, created_at:-1`

backups (0 docs)

Field	Type	Example / note

Indexes: `expires_at:1` TTL

bank_accounts (1 docs)

Field	Type	Example / note
id	uuid (str)	"6c37fcd1-9222-4ad3-b1c6-2d2bdea5ff6a"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.561555+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.561555+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
bank_name	str	"State Bank of India"
account_name	str	"Shree Ganesh Builders Pvt Ltd"
account_number_enc	str	(secret — never returned)
ifsc	str	"SBIN0001234"
branch	str	"Boring Road, Patna"
is_default	bool	true

Indexes: `id:1` unique, `company_id:1, is_active:1, name:1`

block_summaries (5 docs)

Field	Type	Example / note
block_id	uuid (str)	"3d30cc7f-5c79-4d8f-86b9-b3b90578c4a4"
agreement_paise	int	1000000000
bookings	int	1
code	str	"P"
collections_mtd_paise	int	0
collections_total_paise	int	1006000000
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"

Field	Type	Example / note
computed_at	datetime (ISO)	"2026-10-25T08:27:55.805739+00:00"
derived	bool	true
name	str	"Villa Block P (Premium)"
outstanding	object	4 keys
overdue	object	2 keys
paid_paise	int	1006000000
pending_cheques_paise	int	0
progress	object	5 keys
project_id	uuid (str)	"51a6f4a8-b592-4cda-a0d1-e2b0edc870ea"
status	str	"Completed"
units	object	6 keys

Indexes: `block_id:1` unique, `project_id:1`

`block_types` (5 docs)

Field	Type	Example / note
id	uuid (str)	"c86c52c4-4ff6-4375-85d3-de9cdeae0aa7"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.534031+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.534031+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
name	str	"Tower"
code	str	"TOWER"
has_floors	bool	true

Indexes: `id:1` unique, `company_id:1, is_active:1, name:1`

`blocks` (5 docs)

Towers/blocks: floors, floor-plan assignments, skip/refuge/podium, construction stages, legacy milestones.

Field	Type	Example / note
id	uuid (str)	"19eff68e-1f8e-4e43-8f1d-dd572d0e65ed"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.575452+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.575452+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
project_id	uuid (str)	"4c7d684d-358d-4ba6-88d7-d70165368c87"
block_type_id	uuid (str)	"c86c52c4-4ff6-4375-85d3-de9cdeae0aa7"

Field	Type	Example / note
name	str	"Tower A"
code	str	"A"
floors	int	8
flats_per_floor	int	4
floor_start	int	1
numbering_pattern	str	"{block}-{floor}{pos2}"
position_flat_types	list[object]	4 items
floor_plans	list[object]	2 items
skip_floors	list[…]	0 items
refuge_floors	list[…]	0 items
podium_floors	list[…]	0 items
ground_floor_label	null	null
milestone_template_id	uuid (str)	"50880361-4c6f-4004-b826-a73994de3e2d"
rate_per_sqft	int	4200
floor_rise_per_sqft	int	25
status	str	"Under Construction"
construction_percent	int	45
unit_counts	object	7 keys
milestones	list[object]	11 items
construction_stages	list[object]	11 items
construction_stage_sequence	list[object]	11 items
default_cash_percent	null	null

Indexes: `id:1` unique, `project_id:1, code:1` unique, `_fts:text, _ftsx:1`

`bookings` (10 docs)

Agreements: buckets (cash/account), plan snapshot, other charges, revisions, cancellation, drafts.

Field	Type	Example / note
id	uuid (str)	"ebf64193-f49c-4c3b-a7f5-305c9c5b325c"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2025-11-01T10:00:00+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.844679+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
booking_number	str	"BK-2526-0001"
project_id	uuid (str)	"51a6f4a8-b592-4cda-a0d1-e2b0edc870ea"
project_code	str	"GGV"
project_name	str	"Ganesh Green Villas"
block_id	uuid (str)	"3d30cc7f-5c79-4d8f-86b9-b3b90578c4a4"
block_code	str	"P"
block_name	str	"Villa Block P (Premium)"
flat_id	uuid (str)	"7fe350bc-3b21-47dd-87b4-56dae68512bc"

Field	Type	Example / note
customer_id	uuid (str)	"1469b5dc-10c1-4121-99a1-79b6034aa0ab"
customer_name	str	"Amit Kumar Sinha"
customer_code	str	"CUST-2526-0001"
customer_phone	str	"+919431012345"
co_applicant_customer_ids	list[…]	0 items
booking_date	date (ISO)	"2025-11-01"
agreement_value_paise	int	1000000000
cash_agreed_paise	int	600000000
account_agreed_paise	int	400000000
rate_per_sqft	null	null
flat_snapshot	object	6 keys
milestone_template_snapshot	list[object]	10 items
construction_percent	int	100
other_charges	list[object]	1 items
parking_slot_ids	list[uuid (str)]	1 items
sales_user_id	uuid (str)	"c5b75c63-d94a-46a3-be1d-b92b95803990"
lead_id	null	null
broker_name	null	null
remarks	str	"Owner reference ledger example"
status	str	"Completed"
cancellation	null	null
agreement_revisions	list[…]	0 items
summary	object	14 keys
ledger_rebuilt_at	datetime (ISO)	"2026-10-25T08:50:40.753006+00:00"

Indexes: `id:1` unique, `booking_number:1` unique, `project_id:1, status:1`, `block_id:1, status:1`, `customer_id:1`, `flat_id:1`, `company_id:1, status:1, booking_date:-1, id:1`, `company_id:1, customer_id:1, booking_date:-1`, `company_id:1, project_id:1, status:1, booking_date:-1`, `customer_name:1`

`brochure_downloads` (5 docs)

One row per brochure download (sparkline source).

Field	Type	Example / note
id	uuid (str)	"8d1da06d-caa3-4d3a-9c66-6bd28b62dfda"
project_id	uuid (str)	"51a6f4a8-b592-4cda-a0d1-e2b0edc870ea"
lead_id	uuid (str)	"96ea10bd-9575-41c8-83d4-08a8627af057"
at	datetime (ISO)	"2026-10-25T08:46:13.894059+00:00"

Indexes: `lead_id:1, at:-1`, `project_id:1, at:-1`

`brochure_tokens` (0 docs)

Signed-token nonces (TTL index) so a download link counts once.

Field	Type	Example / note

Indexes: `expires_at:1` TTL

cache (0 docs)

Field	Type	Example / note

Indexes: key:1 unique, expires_at:1 TTL

catalogue_items (351 docs)

Field	Type	Example / note
id	uuid (str)	"9b5bef86-e99f-4e27-af11-8679a18651fa"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:55.715284+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:55.715284+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
type	str	"salutation"
code	str	"MR"
label	str	"Mr"
parent_id	null	null
meta	object	0 keys
sort_order	int	0
is_system	bool	true

Indexes: id:1 unique, company_id:1, type:1, code:1 unique, company_id:1, type:1, is_active:1, sort_order:1

charge_types (7 docs)

Field	Type	Example / note
id	uuid (str)	"1e763782-27b5-41a7-b3ea-82baf28669a2"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.550496+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.550496+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
name	str	"Registry Charges"
code	str	"REGISTRY"
default_amount	null	null
is_taxable	bool	false
sort_order	int	1

Indexes: id:1 unique, company_id:1, is_active:1, name:1

cms_banners (3 docs)

Home hero banners.

Field	Type	Example / note
id	uuid (str)	"08e2f88a-3d8d-46a4-89a5-a89dcbdcf440"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:40.075965+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:40.075965+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
order	int	1
image_url	str	"https://images.unsplash.com/photo-1545324418-cc1a3fa10c00?a
headline	str	"Homes that rise with Patna"
subheadline	str	"Shree Ganesh Heights — 2 & 3 BHK residences on Bailey Road
cta_label	str	"Explore Shree Ganesh Heights"
cta_link	str	"/projects/shree-ganesh-heights"
cta2_label	str	"Book a site visit"
cta2_link	str	"/contact"
from_date	null	null
to_date	null	null

Indexes: _id only

cms_gallery (18 docs)

Gallery items.

Field	Type	Example / note
id	uuid (str)	"0bf839b9-c015-4359-a599-9c3f63407428"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:40.078048+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:40.078048+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
order	int	1
project_id	uuid (str)	"4c7d684d-358d-4ba6-88d7-d70165368c87"
category	str	"Exterior"
image_url	str	"https://images.unsplash.com/photo-1545324418-cc1a3fa10c00?a
caption	str	"Tower A & B elevation at dusk"

Indexes: _id only

cms_pages (3 docs)

Legal/static pages.

Field	Type	Example / note
id	uuid (str)	"1fc559c2-9934-4f39-bf77-9b5e0cc0f167"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:40.079950+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:40.079950+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
slug	str	"about"
title	str	"About Shree Ganesh Builders"
body	str	"## Our story\n\nFounded in 1998 by Rajesh Kumar, Shree Gane
sections	object	1 keys

Indexes: _id only

cms_people (3 docs)

Field	Type	Example / note
id	uuid (str)	"2c375b79-2bef-4c59-8455-92a0a457a017"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:40.079137+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:40.079137+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
order	int	1
name	str	"Rajesh Kumar"
designation	str	"Founder & Managing Director"
photo_url	str	"https://images.unsplash.com/photo-1507003211169-0a1dd7228f2
bio	str	"Civil engineer from NIT Patna, Rajesh founded Shree Ganesh

Indexes: _id only

cms_posts (5 docs)

Field	Type	Example / note
id	uuid (str)	"7066d905-36fd-4559-b374-21cae648c6da"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:40.080789+00:00"

Field	Type	Example / note
updated_at	datetime (ISO)	"2026-10-25T08:27:40.080789+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
title	str	"Tower A reaches 60%: what construction-linked payments mean
slug	str	"sgh-tower-a-60-percent"
cover_url	str	"https://images.unsplash.com/photo-1560520653-9e0e4c89eb11?a
excerpt	str	"Tower A at Shree Ganesh Heights has crossed the 60% constru
body	str	"Tower A at Shree Ganesh Heights has crossed the **60% const
tags	list[str]	2 items
author	str	"Shree Ganesh Builders"
status	str	"Published"
publish_at	datetime (ISO)	"2026-10-18T09:00:00+00:00"
seo_title	str	"Tower A reaches 60%: what construction-linked payments mean
seo_description	str	"Tower A at Shree Ganesh Heights has crossed the 60% constru

Indexes: _id only

cms_testimonials (6 docs)

Testimonials.

Field	Type	Example / note
id	uuid (str)	"41b66d94-ee46-4032-b327-1e70b2b8d3a0"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:40.077173+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:40.077173+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
order	int	1
name	str	"Mr Amit Kumar Sinha"
designation	str	"4 BHK Villa, Ganesh Green Villas"
photo_url	str	"https://images.unsplash.com/photo-1507003211169-0a1dd7228f2
quote	str	"The ledger is so transparent — I could see every demand and
rating	int	4
project_id	uuid (str)	"51a6f4a8-b592-4cda-a0d1-e2b0edc870ea"
project_name	str	"Ganesh Green Villas"

Indexes: _id only

communications (0 docs)

Field	Type	Example / note

Indexes: `id:1` unique, `company_id:1, customer_id:1, happened_at:-1`, `company_id:1, lead_id:1, happened_at:-1`

`companies` (1 docs)

Tenant/company master (name, numbering formats, demand due days, GST, bank details).

Field	Type	Example / note
id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.289793+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.289793+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
name	str	"Shree Ganesh Builders"
brand	str	"KKT"
legal_name	str	"Shree Ganesh Builders Pvt. Ltd."
address	object	4 keys
gstin	str	"10AABCS1234F1Z5"
rera_registration	str	"BRERAP00123"
logo_url	null	null
financial_year_start_month	int	4
receipt_prefix	str	"SGB"
bank_account	object	5 keys
authorised_signatory_name	str	"Rajesh Kumar"
booking_number_format	str	"BK-{FY}-{SEQ}"
customer_number_format	str	"CUST-{FY}-{SEQ}"
default_gst_percent	int	18
demand_due_days	int	15
demand_number_format	str	"DM-{FY}-{SEQ}"
letterhead_email	str	"accounts@shreeganeshbuilders.in"
letterhead_phone	str	"+91 612 2234567"
letterhead_tagline	str	"Building trust since 1998"
receipt_number_format	str	"{PREFIX}/{FY}/{SEQ}"

Indexes: `id:1` unique

`construction_photos` (50 docs)

Stage photos with customer visibility.

Field	Type	Example / note
id	uuid (str)	"0fc56517-caf6-4b57-b96b-11bae884a2d0"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:41.745923+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:41.745923+00:00"

Field	Type	Example / note
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
project_id	uuid (str)	"4c7d684d-358d-4ba6-88d7-d70165368c87"
block_id	uuid (str)	"19eff68e-1f8e-4e43-8f1d-dd572d0e65ed"
milestone_seq	int	1
storage_key	str	"construction/4c7d684d-358d-4ba6-88d7-d70165368c87/19eff68e-
mime	str	"image/jpeg"
size	int	296431
caption	str	"Tower A — Booking"
taken_on	date (ISO)	"2025-11-01"
visible_to_customers	bool	true
show_on_website	bool	false
order	int	1
url	str	"/api/public/media/photo/0fc56517-caf6-4b57-b96b-11bae884a2d

Indexes: _id only

construction_stages (16 docs)

Construction stage master sequence.

Field	Type	Example / note
id	uuid (str)	"5711a33f-c369-48ac-b5f4-212452a87d33"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
name	str	"Excavation"
code	str	"EXCAVATION"
description	null	null
seq_hint	int	10
is_possession	bool	false
applies_to_block_types	list[…]	0 items
is_active	bool	true
version	int	1
created_at	datetime (ISO)	"2026-10-25T08:27:55.545887+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:55.545891+00:00"
created_by	str	"system"

Indexes: name:1 unique

contra_entries (1 docs)

Field	Type	Example / note
id	uuid (str)	"c55a9833-735c-4bbf-a0ee-0ee56757bbbd"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:40.063432+00:00"

Field	Type	Example / note
updated_at	datetime (ISO)	"2026-10-25T08:27:40.063432+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
date	date (ISO)	"2025-11-05"
kind	str	"DEPOSIT"
amount_paise	int	50000000
bank_account_id	uuid (str)	"6c37fcd1-9222-4ad3-b1c6-2d2bdea5ff6a"
bank_name	str	"State Bank of India"
remarks	str	"Cash deposited at SBI Boring Road"
by_name	str	"Amit Verma"

Indexes: `id:1` unique, `date:1`

`counters` (9 docs)

Atomic sequences per FY/kind + round-robin pointer.

Field	Type	Example / note
scope	str	"customer_number"
key	str	"2526"
seq	int	12

Indexes: `scope:1, key:1` unique

`customer_summaries` (12 docs)

Field	Type	Example / note
customer_id	uuid (str)	"2fb29c1e-38ec-48d1-a11e-e9e3f2392efb"
agreement_paise	int	710000000
bookings	int	1
brochure_downloads	int	0
collections_mtd_paise	int	0
collections_total_paise	int	177500000
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
computed_at	datetime (ISO)	"2026-10-25T08:27:55.863562+00:00"
customer_code	str	"CUST-2526-0007"
derived	bool	true
due_paise	int	77500000
engagement_score	int	15
full_name	str	"Manoj Tiwari"
last_contacted_at	null	null
last_payment_at	date (ISO)	"2026-02-20"
lifecycle_stage	str	"Customer"
outstanding	object	4 keys
overdue	object	2 keys

Field	Type	Example / note
overdue_paise	int	77500000
paid_paise	int	177500000
pending_cheques_paise	int	0
visits	int	0

Indexes: customer_id:1 unique, company_id:1, engagement_score:-1

customers (12 docs)

KYC-sealed customer master (PAN/Aadhaar encrypted + hashed), portal account flags.

Field	Type	Example / note
id	uuid (str)	"1469b5dc-10c1-4121-99a1-79b6034aa0ab"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.779893+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.779893+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
customer_code	str	"CUST-2526-0001"
salutation	str	"Mr"
full_name	str	"Amit Kumar Sinha"
father_or_spouse_name	str	"Ramesh Sinha"
dob	date (ISO)	"1970-01-15"
gender	str	"Male"
phone	str	"+919431012345"
alt_phone	null	null
email	str	"amit.sinha@example.com"
address_current	object	4 keys
address_permanent	object	4 keys
permanent_same_as_current	bool	true
occupation	str	"Business"
company_name	null	null
annual_income_band	str	"10-25L"
source	str	"Referral"
assigned_sales_user_id	uuid (str)	"c5b75c63-d94a-46a3-be1d-b92b95803990"
bank	object	3 keys
nominee	object	3 keys
co_applicants	list[object]	1 items
kyc_status	str	"Verified"
kyc_verified_by	uuid (str)	"3b167223-4312-437a-83b8-2d2483ebefbc"
kyc_verified_at	datetime (ISO)	"2025-11-01T10:00:00+00:00"
notes	null	null
tags	list[…]	0 items

Field	Type	Example / note
pan_enc	str	(secret — never returned)
pan_hash	str	"13fa00c2b994cb26eedb53467cb3c711819dabdd8ab85d87327416265d0
aadhaar_enc	str	(secret — never returned)
engagement_score	int	3
lifecycle_stage	str	"Owner"
gender_id	uuid (str)	"f51c905d-61f8-4e5d-b450-823185f7ca2f"
income_band_id	uuid (str)	"5366fa54-b585-47a7-8152-117717f8f3d3"

Indexes: id:1 unique, company_id:1, phone:1, pan_hash:1, customer_code:1 unique, _fts:text, _ftsx:1, company_id:1, is_active:1, created_at:-1, id:1, company_id:1, full_name:1, company_id:1, lifecycle_stage:1, engagement_score:-1, company_id:1, assigned_sales_user_id:1, next_follow_up_at:1

dashboard_snapshots (1 docs)

Field	Type	Example / note
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
scope_hash	str	"*"
computed_at	datetime (ISO)	"2026-10-25T08:45:27.975998+00:00"
derived	bool	true
payload	object	3 keys
scope	list[str]	1 items

Indexes: company_id:1, scope_hash:1 unique

demands (53 docs)

Installment demands raised by the dues engine (one per booking+installment_seq).

Field	Type	Example / note
id	uuid (str)	"0473765a-b6cf-4cfd-bdcb-9d98cd3f35bd"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
booking_id	uuid (str)	"ebf64193-f49c-4c3b-a7f5-305c9c5b325c"
customer_id	uuid (str)	"1469b5dc-10c1-4121-99a1-79b6034aa0ab"
project_id	uuid (str)	"51a6f4a8-b592-4cda-a0d1-e2b0edc870ea"
block_id	uuid (str)	"3d30cc7f-5c79-4d8f-86b9-b3b90578c4a4"
flat_id	uuid (str)	"7fe350bc-3b21-47dd-87b4-56dae68512bc"
demand_number	str	"DM-2526-0001"
milestone_seq	int	1
milestone_name	str	"Booking"
cumulative_percent	int	10
cash_demand_paise	int	60000000
account_demand_paise	int	40000000
total_demand_paise	int	100000000
demand_date	date (ISO)	"2025-11-01"
due_date	date (ISO)	"2025-11-16"
status	str	"Paid"

Field	Type	Example / note
paid_paise	int	100000000
voided	bool	false
posted_at	datetime (ISO)	"2025-11-01T09:30:00+00:00"
created_at	datetime (ISO)	"2026-10-25T08:27:39.815818+00:00"
installment_name	str	"Booking"
installment_seq	int	1
percent	float	10.0
trigger	str	"On booking"
trigger_stage_id	null	null
trigger_type	str	"ON_BOOKING"

Indexes: id:1 unique, booking_id:1, posted_at:1, block_id:1, milestone_seq:1, company_id:1, status:1, due_date:1, booking_id:1, installment_seq:1 unique partial, customer_id:1, status:1, due_date:1, project_id:1, status:1, due_date:1 partial

document_types (6 docs)

Field	Type	Example / note
id	uuid (str)	"81ba395c-25ba-462a-84aa-fc12ece1119f"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.562586+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.562586+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
name	str	"PAN Card"
code	str	"PAN"
applies_to	str	"customer"

Indexes: id:1 unique, company_id:1, is_active:1, name:1

documents (5 docs)

Uploaded files metadata (object-storage key, mime, size, sharing flags).

Field	Type	Example / note
id	uuid (str)	"59d672e8-c33d-42f1-af9e-f7a33ce52568"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:55.157448+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:55.157448+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
entity_type	str	"booking"
entity_id	uuid (str)	"ebf64193-f49c-4c3b-a7f5-305c9c5b325c"

Field	Type	Example / note
document_type_id	null	null
document_type_name	str	"Allotment letter"
title	str	"Allotment letter BK-2526-0001"
file_name	str	"allotment-BK-2526-0001.pdf"
mime	str	"application/pdf"
size	int	4820
storage_key	str	"documents/7ee62ac6-926d-471f-8c7e-f7c30848bb86/booking/ebf6
is_sensitive	bool	false
shared_with_customer	bool	true
uploaded_by	null	null

Indexes: `id:1` unique, `entity_type:1, entity_id:1`

`flat_types` (5 docs)

Field	Type	Example / note
id	uuid (str)	"d1495095-a96d-4270-adb0-dd2fe4c2a5e6"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.567546+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.567546+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
name	str	"2BHK-A"
code	str	"2BHK-A"
bhk	int	2
facing	str	"E"
description	str	"Compact 2 BHK with balcony"
rooms	list[object]	7 items
areas	list[object]	4 items
base_rate_per_sqft	int	4000
base_price	null	null
spec_item_ids	list[uuid (str)]	10 items

Indexes: `id:1` unique, `company_id:1, is_active:1, name:1`

`flats` (138 docs)

Units generated per block: number, floor, type snapshot, areas, price breakup, status, booking link.

Field	Type	Example / note
id	uuid (str)	"1d2c16d7-ff12-4ebb-8dd3-4d40060402c1"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.578931+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.605417+00:00"

Field	Type	Example / note
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
project_id	uuid (str)	"4c7d684d-358d-4ba6-88d7-d70165368c87"
block_id	uuid (str)	"19eff68e-1f8e-4e43-8f1d-dd572d0e65ed"
flat_type_id	uuid (str)	"9a11307a-f6eb-428b-8d7c-4dc5f1af1b58"
flat_type_code	str	"3BHK-A"
flat_number	str	"A-101"
floor	int	1
floor_label	str	"1"
position	int	1
facing	str	"N"
is_corner	bool	true
status	str	"Hold"
rooms_snapshot	list[object]	10 items
areas_snapshot	list[object]	4 items
carpet_sqft	int	950
builtup_sqft	int	1100
super_builtup_sqft	int	1450
rate_per_sqft	int	4200
price	int	6090000
price_breakup	list[object]	1 items
rate_card_id	null	null
notes	null	null
status_changed_at	datetime (ISO)	"2026-10-25T08:27:39.605410+00:00"
status_reason	str	"Reserved for channel partner"

Indexes: `id:1` unique, `block_id:1, flat_number:1` unique, `project_id:1, status:1`, `block_id:1, floor:1, position:1`, `_fts:text, _ftsx:1`, `flat_number:1, company_id:1, status:1, project_id:1`

`floor_plan_templates` (4 docs)

Field	Type	Example / note
id	uuid (str)	"f5843977-716d-463e-b7d0-e0030790e66c"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.633867+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.633867+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
name	str	"Typical A"
code	str	"TYP-A"
flats_per_floor	int	4

Field	Type	Example / note
positions	list[object]	4 items

Indexes: id:1 unique, company_id:1, is_active:1, name:1

idempotency_keys (0 docs)

Idempotency-Key → cached response for POSTs.

Field	Type	Example / note
user_id	str	"35.225.230.28,172.70.194.157,34.149.9.106"
key	uuid (str)	"aa6ec2f7-45cd-468e-b1db-c94ffed64756"
path	str	"/api/auth/login"
status	int	200
body	str	"{\"access_token\":\"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.ey
created_at	datetime	"2026-10-25 08:29:45.915000"

Indexes: user_id:1, key:1 unique, created_at:1 TTL

import_batches (0 docs)

Field	Type	Example / note

Indexes: _id only

leads (8 docs)

CRM leads with stage history, follow-ups, activities timeline, assignment.

Field	Type	Example / note
id	uuid (str)	"ddfe57d0-da32-4c4b-b2dc-dc8d09d0b9aa"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.803536+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:46:13.886460+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
lead_number	str	"LD-2627-0001"
name	str	"Arvind Kumar"
phone	str	"+919430011111"
email	str	"arvind@example.com"
project_id	uuid (str)	"4c7d684d-358d-4ba6-88d7-d70165368c87"
flat_type_id	null	null
budget_min	int	4000000
budget_max	int	7000000
source	str	"Website"
status	str	"New"
stage_history	list[object]	1 items
assigned_to	uuid (str)	"c5b75c63-d94a-46a3-be1d-b92b95803990"

Field	Type	Example / note
notes	null	null
follow_ups	list[object]	1 items
next_follow_up_at	datetime (ISO)	"2026-10-25T11:00:00+00:00"
lost_reason	null	null
converted_customer_id	null	null
activities	list[object]	1 items

Indexes: id:1 unique, company_id:1, status:1, next_follow_up_at:1, company_id:1, source:1, is_read:1, company_id:1, status:1, created_at:-1, id:1, company_id:1, project_id:1, status:1, company_id:1, phone:1, company_id:1, assigned_to:1, next_follow_up_at:1

ledger_entries (2982 docs)

Field	Type	Example / note
id	uuid (str)	"d273ee1a-5597-4f96-96eb-87a3187ab2f7"
booking_id	uuid (str)	"ebf64193-f49c-4c3b-a7f5-305c9c5b325c"
seq	int	1
at	datetime (ISO)	"2025-11-01T09:30:00+00:00"
date	date (ISO)	"2025-11-01"
type	str	"DEMAND"
construction_percent	int	10
category	null	null
amount_paise	int	100000000
note	str	"Booking (10.0%)"
ref	str	"DM-2526-0001"
cash_paid_cum_paise	int	0
account_paid_cum_paise	int	0
cash_order_raised_paise	int	60000000
account_order_raised_paise	int	40000000
cash_due_paise	int	60000000
account_due_paise	int	40000000
total_due_paise	int	100000000
other_charges_due_paise	int	6000000

Indexes: id:1 unique, booking_id:1, seq:1, booking_id:1, ref:1

locks (1 docs)

Field	Type	Example / note
name	str	"scheduler"
expires_at	datetime	"2026-10-25 08:51:25.550000"
owner	str	"agent-env-a2ad8b1d-236b-431e-8c34-6627548d7c1b:10914"

Indexes: name:1 unique

login_events (1398 docs)

Field	Type	Example / note
id	uuid (str)	"f6e3aee2-cc3a-4d13-bebd-3b98191ddc23"
email	str	"admin@kkt.in"
user_id	uuid (str)	"e6d1f80b-4ffa-40c7-aeda-15f66372e1a2"
success	bool	true
reason	str	"ok"
ip	str	"127.0.0.1"
user_agent	str	"curl/7.88.1"
created_at	datetime (ISO)	"2026-10-23T22:58:28.400409+00:00"

Indexes: `created_at:-1`, `email:1`, `account_id:1, success:1, created_at:-1`

`milestone_templates` (2 docs)

Field	Type	Example / note
id	uuid (str)	"50880361-4c6f-4004-b826-a73994de3e2d"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.548719+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.548719+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
name	str	"Standard CLP"
milestones	list[object]	10 items

Indexes: `id:1` unique, `company_id:1, is_active:1, name:1`

`notices` (2 docs)

Field	Type	Example / note
id	uuid (str)	"5fccec67-d11d-4358-90ee-f0e2c57241c4"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:40.548019+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:40.548019+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
title	str	"Diwali office hours & site visit slots"
body	str	"Our offices will remain open **10 am – 4 pm** during the fe
project_ids	list[…]	0 items
project_names	list[…]	0 items
publish_at	datetime (ISO)	"2026-10-25T08:27:40.548027+00:00"
expires_at	null	null
pinned	bool	true

Indexes: `_id` only

`notifications` (67 docs)

ERP user notifications.

Field	Type	Example / note
id	uuid (str)	"36c2e16e-aac7-42fe-8a23-184eda88418e"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:54.985772+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:54.985801+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
user_id	uuid (str)	"c5b75c63-d94a-46a3-be1d-b92b95803990"
title	str	"New service request"
body	str	"SR-2627-0001 · Receipt for NEFT not received · Priya Verma"
kind	str	"service.create"
entity_type	str	"service_request"
entity_id	uuid (str)	"0c3779a9-5867-465d-936b-19e0adaf14a2"
link	str	"/service-requests/0c3779a9-5867-465d-936b-19e0adaf14a2"
read	bool	false

Indexes: `user_id:1, read:1, created_at:-1`, `user_id:1, withdrawn:1, read:1`

`parking_slots` (132 docs)

Parking inventory & allotment.

Field	Type	Example / note
id	uuid (str)	"1c15cc57-f98b-478e-a3ba-f9a34be76c8d"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.742671+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.742671+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
project_id	uuid (str)	"4c7d684d-358d-4ba6-88d7-d70165368c87"
number	str	"SB001"
type	str	"Basement"
level	str	"B1"
price	int	350000
status	str	"Allotted"
flat_id	null	null
booking_id	uuid (str)	"9aa47f45-1cef-4160-8fc7-2edf6ce1bd5b"
customer_id	uuid (str)	"f0cdddf8-1e69-4213-8ddc-e8a24920d075"

Indexes: `id:1` unique, `project_id:1, number:1` unique, `project_id:1, status:1`

payment_modes (6 docs)

Field	Type	Example / note
id	uuid (str)	"59e872ad-fadb-4ac1-96c5-0078b3d71a69"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.556277+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.556277+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
name	str	"Cash"
code	str	"CASH"
requires_reference	bool	false

Indexes: id:1 unique, company_id:1, is_active:1, name:1

payment_plan_assignments (7 docs)

Plan ↔ project/tower assignment (default flag).

Field	Type	Example / note
block_id	null	null
project_id	uuid (str)	"51a6f4a8-b592-4cda-a0d1-e2b0edc870ea"
payment_plan_id	uuid (str)	"4de6f977-e9f9-4707-9f65-5e797388893b"
block_name	null	null
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:55.554369+00:00"
created_by	str	"system"
effective_from	null	null
effective_to	null	null
id	uuid (str)	"890348dd-6ddd-46c6-8921-7891685f29d6"
is_active	bool	true
is_default	bool	true
plan_code	str	"VILLA_CLP"
plan_name	str	"Villa CLP"
project_name	str	"Ganesh Green Villas"

Indexes: project_id:1, block_id:1, is_active:1

payment_plans (4 docs)

Payment plan masters with installments and triggers.

Field	Type	Example / note
id	uuid (str)	"2cd9d74d-6c4f-41b7-8e3d-ff0c6c19dd9c"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
name	str	"Standard CLP"
code	str	"STANDARD_CLP"
description	str	"Converted from milestone template 'Standard CLP'"

Field	Type	Example / note
plan_type	str	"CLP"
installments	list[object]	10 items
applies_percent_to	str	"BOTH_BUCKETS"
grace_days	int	0
is_active	bool	true
version	int	1
created_at	datetime (ISO)	"2026-10-25T08:27:55.551433+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:55.551437+00:00"
created_by	str	"system"
legacy_template_id	uuid (str)	"50880361-4c6f-4004-b826-a73994de3e2d"

Indexes: `name:1` unique

`payments` (0 docs)

Receipts: amount in paise, category, mode, clearing status, reversal.

Field	Type	Example / note
id	uuid (str)	"23149fc9-e91d-4062-8ea2-fc0f5603f905"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2025-11-03T14:00:00+00:00"
updated_at	datetime (ISO)	"2025-11-03T14:00:00+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
receipt_number	str	"SGB/2526/00001"
booking_id	uuid (str)	"ebf64193-f49c-4c3b-a7f5-305c9c5b325c"
booking_number	str	"BK-2526-0001"
customer_id	uuid (str)	"1469b5dc-10c1-4121-99a1-79b6034aa0ab"
customer_name	str	"Amit Kumar Sinha"
project_id	uuid (str)	"51a6f4a8-b592-4cda-a0d1-e2b0edc870ea"
project_code	str	"GGV"
block_id	uuid (str)	"3d30cc7f-5c79-4d8f-86b9-b3b90578c4a4"
block_code	str	"P"
flat_id	uuid (str)	"7fe350bc-3b21-47dd-87b4-56dae68512bc"
flat_number	str	"P1"
against	str	"FLAT_VALUE"
payment_category	str	"Account"
other_charge_line_id	null	null
other_charge_name	null	null
amount_paise	int	40000000
payment_date	date (ISO)	"2025-11-03"
payment_mode_id	uuid (str)	"1893c2d8-a054-43f9-bd75-a92c2c0db4a9"
payment_mode_code	str	"NEFT_RTGS"
payment_mode_name	str	"NEFT / RTGS"

Field	Type	Example / note
instrument	object	4 keys
bank_account_id	uuid (str)	"6c37fcd1-9222-4ad3-b1c6-2d2bdea5ff6a"
bank_account_name	str	"State Bank of India **7890"
remarks	null	null
received_by	uuid (str)	"3b167223-4312-437a-83b8-2d2483ebefbc"
received_by_name	str	"Amit Verma"
status	str	"Cleared"
cleared_at	datetime (ISO)	"2025-11-03T14:00:00+00:00"
bounced_at	null	null
reversal	null	null
idempotency_key	null	null
posted_at	datetime (ISO)	"2025-11-03T14:00:00+00:00"

Indexes: id:1 unique, receipt_number:1 unique, booking_id:1, status:1, company_id:1, payment_date:-1, project_id:1, payment_date:-1, company_id:1, status:1, payment_date:-1, id:1, booking_id:1, against:1, payment_category:1, status:1, customer_id:1, company_id:1, status:1, payment_mode_code:1 partial

plc_types (6 docs)

Field	Type	Example / note
id	uuid (str)	"3d5603cc-280e-4893-b17b-bc0650eae2d0"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.663596+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.663596+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
name	str	"Floor Rise"
code	str	"FLOOR_RISE"
calc	str	"per_sqft"
description	null	null
sort_order	int	1

Indexes: id:1 unique, company_id:1, is_active:1, name:1

portal_accounts (12 docs)

Field	Type	Example / note
id	uuid (str)	"54d56d9e-af0e-41cb-915e-48b96e96795a"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:40.537334+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:40.537334+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null

Field	Type	Example / note
version	int	1
customer_id	uuid (str)	"b0a57aa2-aa0f-43bc-bca9-e7a14bd09120"
login_phone	str	"9431001234"
password_hash	str	(secret — never returned)
must_change_password	bool	false
failed_attempts	int	0
locked_until	null	null
last_login_at	null	null
read_notice_ids	list[…]	0 items

Indexes: customer_id:1

portal_refresh_tokens (1 docs)

Field	Type	Example / note
id	str	"edb64b508d0f967b"
account_id	uuid (str)	"afd26ae9-c026-4e76-9ca7-6ce5dd444bc7"
family	str	"85ad8898f98a084f"
token_hash	str	(secret — never returned)
revoked	bool	false
ip	str	"35.225.230.28"
ua	str	"Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,
created_at	datetime (ISO)	"2026-10-25T08:30:54.498156+00:00"
expires_at	datetime (ISO)	"2026-11-24T08:30:54.498162+00:00"

Indexes: _id only

project_summaries (2 docs)

Field	Type	Example / note
project_id	uuid (str)	"51a6f4a8-b592-4cda-a0d1-e2b0edc870ea"
agreement_paise	int	2100000000
blocks	list[object]	2 items
bookings	int	2
city	str	"Patna"
code	str	"GGV"
collections_mtd_paise	int	0
collections_total_paise	int	1981000000
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
computed_at	datetime (ISO)	"2026-10-25T08:38:27.924985+00:00"
cover_image_url	null	null
derived	bool	true
leads_open	int	3
name	str	"Ganesh Green Villas"
outstanding	object	4 keys
overdue	object	2 keys

Field	Type	Example / note
paid_paise	int	1831000000
pending_cheques_paise	int	0
progress_pct	float	83.5
status	str	"Planning"
towers	int	2
units	object	6 keys
visits_upcoming	int	0

Indexes: `project_id:1` unique, `company_id:1, project_id:1`

`projects` (2 docs)

Projects incl. address, RERA, unit counts, `website` sub-document (CMS/SEO/brochure gate/site-visit settings).

Field	Type	Example / note
id	uuid (str)	"4c7d684d-358d-4ba6-88d7-d70165368c87"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.572224+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.572224+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
name	str	"Shree Ganesh Heights"
code	str	"SGH"
description	str	"Premium high-rise residential towers in the heart of Patna
project_type	str	"Residential"
address	object	4 keys
rera_number	str	"BRERAP00123"
launch_date	date (ISO)	"2025-10-15"
expected_possession_date	date (ISO)	"2028-12-31"
status	str	"Under Construction"
amenity_ids	list[uuid (str)]	12 items
milestone_template_id	uuid (str)	"50880361-4c6f-4004-b826-a73994de3e2d"
cover_image_url	null	null
default_cash_percent	int	60
is_published	bool	false
show_prices	bool	false
show_availability	bool	false
block_count	int	3
unit_counts	object	7 keys
approvals	list[object]	3 items
escrow_bank_account_id	uuid (str)	"6c37fcd1-9222-4ad3-b1c6-2d2bdea5ff6a"
geo	object	2 keys

Field	Type	Example / note
land_area_sqft	int	217800
landmarks	list[object]	3 items
phases	list[object]	2 items
team	list[uuid (str)]	2 items
website	object	19 keys
construction_stage_sequence	list[object]	9 items

Indexes: id:1 unique, company_id:1, code:1 unique, _fts:text, _ftsx:1

rate_cards (2 docs)

Rate cards per project/block.

Field	Type	Example / note
id	uuid (str)	"d3dde93f-e352-4909-8724-0ada3c1f9245"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.668754+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.668754+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
project_id	uuid (str)	"4c7d684d-358d-4ba6-88d7-d70165368c87"
name	str	"SGH Launch Rate Card FY26"
effective_from	date (ISO)	"2026-04-01"
rates	list[object]	4 items
plc_rules	list[object]	3 items
notes	str	"Seeded"
status	str	"Active"
activated_at	datetime (ISO)	"2026-10-25T08:27:39.668746+00:00"
superseded_by	null	null
applied_at	datetime (ISO)	"2026-10-25T08:27:39.668751+00:00"
applied_count	int	115

Indexes: id:1 unique, project_id:1, status:1

rate_limits (4 docs)

Mongo-backed sliding-window counters.

Field	Type	Example / note
slot	int	29881969
bucket	str	"login:127.0.0.1"
count	int	1
expires_at	datetime	"2026-10-25 08:51:21.648000"

Indexes: bucket:1, slot:1 unique, expires_at:1 TTL

`refresh_tokens` (9 docs)

Rotating refresh tokens (family id, reuse detection).

Field	Type	Example / note
`id`	uuid (str)	"6a27c696-2fec-4374-af88-f6003be1427b"
`user_id`	uuid (str)	"6396ea03-7ba8-4f48-bbac-763d02103836"
`company_id`	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
`token_hash`	str	(secret — never returned)
`revoked`	bool	false
`ip`	str	"127.0.0.1"
`user_agent`	str	"curl/7.88.1"
`created_at`	datetime (ISO)	"2026-10-25T08:28:32.221461+00:00"
`last_seen_at`	datetime (ISO)	"2026-10-25T08:28:32.221464+00:00"
`expires_at`	datetime (ISO)	"2026-11-01T08:28:32.221466+00:00"

Indexes: `token_hash:1` unique, `user_id:1`

`report_cache` (0 docs)

Field	Type	Example / note

Indexes: `expires_at:1` TTL

`report_presets` (0 docs)

Field	Type	Example / note

Indexes: `user_id:1, report_key:1`

`roles` (6 docs)

System + custom roles with permission lists.

Field	Type	Example / note
`id`	uuid (str)	"0780b8f8-55e9-4c4d-b4e8-5db830467d36"
`company_id`	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
`created_at`	datetime (ISO)	"2026-10-25T08:27:39.291109+00:00"
`updated_at`	datetime (ISO)	"2026-10-25T08:27:39.291109+00:00"
`created_by`	str	"system"
`is_active`	bool	true
`deleted_at`	null	null
`version`	int	1
`name`	str	"Super Admin"
`description`	str	"Full access to everything"
`permissions`	list[str]	36 items
`is_system`	bool	true

Indexes: `id:1` unique, `company_id:1, name:1`

`room_types` (8 docs)

Field	Type	Example / note
id	uuid (str)	"61482341-5915-43fd-a80c-cca49b6113a7"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.520558+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.520558+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
name	str	"Living Room"
code	str	"LIVING"
icon	str	"sofa"
sort_order	int	1

Indexes: `id:1` unique, `company_id:1, is_active:1, name:1`

`service_categories` (7 docs)

Field	Type	Example / note
id	uuid (str)	"894ccd10-7f3d-4083-9e00-acdab44b0ca0"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:40.545955+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:40.545955+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
name	str	"Payment query"
order	int	0

Indexes: `_id` only

`service_requests` (5 docs)

Customer service tickets with SLA, notes, attachments, rating.

Field	Type	Example / note
id	uuid (str)	"0c3779a9-5867-465d-936b-19e0adaf14a2"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:54.984026+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:54.984026+00:00"
created_by	null	null
is_active	bool	true
deleted_at	null	null
version	int	1
ticket_number	str	"SR-2627-0001"
customer_id	uuid (str)	"f0cdddf8-1e69-4213-8ddc-e8a24920d075"
customer_name	str	"Priya Verma"

Field	Type	Example / note
customer_phone	str	"+919431023456"
booking_id	uuid (str)	"9aa47f45-1cef-4160-8fc7-2edf6ce1bd5b"
booking_number	str	"BK-2526-0002"
project_id	uuid (str)	"4c7d684d-358d-4ba6-88d7-d70165368c87"
project_name	str	"Shree Ganesh Heights"
block_id	uuid (str)	"19eff68e-1f8e-4e43-8f1d-dd572d0e65ed"
flat_number	str	"A-201"
category	str	"Payment query"
priority	str	"High"
subject	str	"Receipt for NEFT not received"
description	str	"I transferred the Plinth demand via NEFT on 3rd; please iss
attachments	list[…]	0 items
status	str	"In Progress"
assigned_to	uuid (str)	"c5b75c63-d94a-46a3-be1d-b92b95803990"
sla_due_at	datetime (ISO)	"2026-10-26T08:27:54.984059+00:00"
sla_breached	bool	false
rating	null	null
resolved_at	null	null
closed_at	null	null
thread	list[object]	3 items
stage_history	list[object]	2 items

Indexes: `company_id:1, status:1, created_at:-1`, `customer_id:1, created_at:-1`

`settings` (18 docs)

Key/value system settings (roles_version, seed markers, site settings).

Field	Type	Example / note
key	str	"roles_version"
value	int	16

Indexes: `key:1` unique

`site_settings` (1 docs)

Public site settings (brand, phone, WhatsApp, offices, trust stats, round-robin).

Field	Type	Example / note
id	uuid (str)	"f927ce08-3472-484f-bec1-a9540d765f6b"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:40.075103+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:40.075103+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1

Field	Type	Example / note
brand_name	str	"Shree Ganesh Builders"
tagline	str	"Crafting landmark homes in Patna since 1998"
logo_url	null	null
favicon_url	null	null
phone	str	"+916122234567"
whatsapp	str	"+919430012345"
email	str	"sales@shreeganeshbuilders.in"
offices	list[object]	2 items
social	object	4 keys
trust_stats	list[object]	4 items
footer_text	str	"Shree Ganesh Builders Pvt. Ltd. is a RERA-registered real e
rera_disclaimer	str	"All projects are registered under the Bihar Real Estate Reg
seo_title	str	"Shree Ganesh Builders — Premium Apartments & Villas in Patn
seo_description	str	"RERA-registered builder in Patna. Explore Shree Ganesh Heig
og_image_url	str	"https://images.unsplash.com/photo-1545324418-cc1a3fa10c00?a
ga_id	null	null
meta_pixel_id	null	null
round_robin_assignment	bool	true
site_live	bool	true
portal_url	str	"https://royal-realty-hub-2.expo.preview.emergentagent.com"
home	object	4 keys

Indexes: _id only

site_visits (10 docs)

Site-visit bookings (SV numbers, slot, capacity, status, timeline).

Field	Type	Example / note
id	uuid (str)	"c82508b4-c361-4947-8d6f-32ebdb5fe35e"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:46:13.882278+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:46:13.882278+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
visit_number	str	"SV-2627-0001"
lead_id	uuid (str)	"96ea10bd-9575-41c8-83d4-08a8627af057"
customer_id	null	null
project_id	uuid (str)	"51a6f4a8-b592-4cda-a0d1-e2b0edc870ea"
project_name	str	"Ganesh Green Villas"
date	date (ISO)	"2026-10-31"
slot_start	str	"11:00"
slot_end	str	"12:00"

Field	Type	Example / note
name	str	"Pooja Sharma"
phone	str	"+919430022222"
email	str	"pooja@example.com"
visitors_count	int	1
status	str	"Confirmed"
assigned_to	uuid (str)	"c5b75c63-d94a-46a3-be1d-b92b95803990"
source	str	"ERP"
notes	null	null
timeline	list[object]	1 items

Indexes: `project_id:1, date:1, slot_start:1`, `company_id:1, date:1, slot_start:1`, `project_id:1, date:1, status:1`, `lead_id:1, status:1, date:1`

`spec_categories` (6 docs)

Field	Type	Example / note
id	uuid (str)	"21bbc6af-2f2e-4426-a549-4d88c1d68782"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.749575+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.749575+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
name	str	"Flooring"
code	str	"FLOORING"
sort_order	int	1

Indexes: `id:1` unique, `company_id:1, is_active:1, name:1`

`spec_items` (12 docs)

Field	Type	Example / note
id	uuid (str)	"12c87f3a-9df0-4bcc-91ed-3c12787cbb6f"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.753824+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.753824+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
name	str	"Vitrified tiles 800×800"
spec_category_id	uuid (str)	"21bbc6af-2f2e-4426-a549-4d88c1d68782"
description	null	null
brand	str	"Kajaria"

Indexes: `id:1` unique, `company_id:1, is_active:1, name:1`

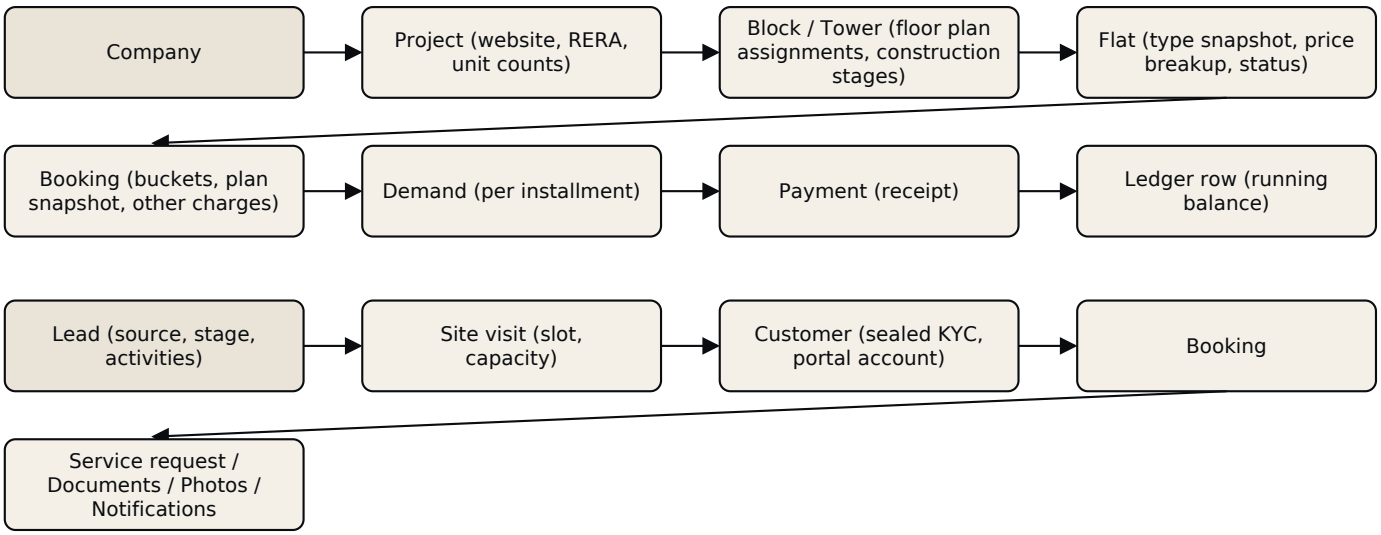
`users` (6 docs)

ERP users with role, project scope, password hash/history, MFA, lockout counters.

Field	Type	Example / note
id	uuid (str)	"6396ea03-7ba8-4f48-bbac-763d02103836"
company_id	uuid (str)	"7ee62ac6-926d-471f-8c7e-f7c30848bb86"
created_at	datetime (ISO)	"2026-10-25T08:27:39.617684+00:00"
updated_at	datetime (ISO)	"2026-10-25T08:27:39.617684+00:00"
created_by	str	"system"
is_active	bool	true
deleted_at	null	null
version	int	1
name	str	"Rajesh Kumar"
email	str	"admin@kkt.in"
phone	null	null
role_id	uuid (str)	"0780b8f8-55e9-4c4d-b4e8-5db830467d36"
project_scope	list[str]	1 items
password_hash	str	(secret — never returned)
failed_attempts	int	0
locked_until	null	null
mfa_enabled	bool	false
must_change_password	bool	false
password_history	list[…]	0 items
last_login_at	datetime (ISO)	"2026-10-25T08:50:56.051474+00:00"

Indexes: `email:1` unique, `id:1` unique, `_fts:text, _ftsx:1`

4.1 Core entity relationships



Masters referenced by id and snapshotted into the dependent document at write time: flat types, floor-plan templates, PLC types, charge types, amenities, banks, document types, construction stage masters, payment plans. Snapshots (`flat_type_snapshot`, `payment_plan_snapshot`, `construction_stages` copied into the block) make history immutable even when a master changes

later.

4.2 State machines

Flat



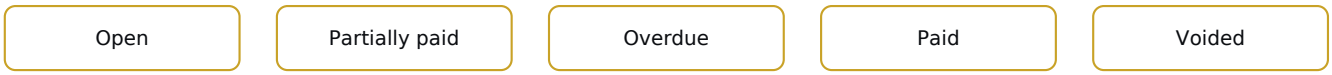
From	To	Transition / guard
Available	Hold	sales places a hold (expires automatically after hold_hours)
Hold	Available	hold released or expired
Available	Booked	booking confirmed (atomic find_one_and_update on status ∈ {Available, Hold})
Hold	Booked	same precondition
Booked	Sold	handover recorded on the booking
Booked	Available	booking cancelled (refund computed)
Available	Blocked	admin blocks the unit (not sellable)
Blocked	Available	unblock

Booking



From	To	Transition / guard
Draft	Active	wizard confirm (flat lock + agreement + plan snapshot + catch-up demands)
Active	Cancelled	cancellation with refund ≤ paid (REFUND_EXCEEDS_PAID)
Active	Completed	handover; flat → Sold

Demand



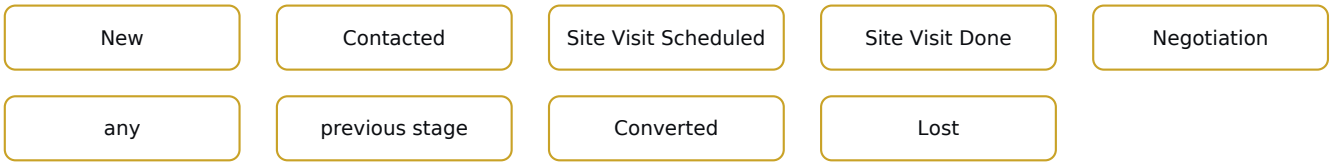
From	To	Transition / guard
Open	Partially paid	cleared payment allocated below total
Open	Overdue	due_date < today and unpaid (hourly sweep)
Partially paid	Paid	paid_paise ≥ total_demand_paise
Open	Voided	stage undo / integrity repair (unpaid only)

Payment



From	To	Transition / guard
Pending	Cleared	cheque/DD cleared (clearing endpoint); CASH/UPI/NEFT_RTGS/LOAN_DISBURSEMENT clear immediately
Pending	Bounced	cheque returned; ledger unaffected
Cleared	Reversed	reversal with reason; ledger row REVERSAL appended

Lead



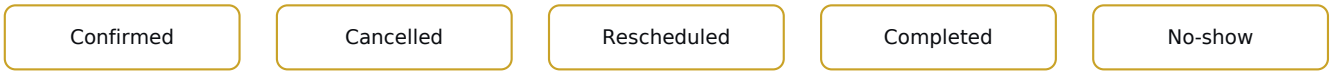
From	To	Transition / guard
New	Contacted	first follow-up logged
Contacted	Site Visit Scheduled	visit booked (also New → Site Visit Scheduled)
Site Visit Scheduled	Site Visit Done	visit marked Completed
Site Visit Scheduled	previous stage	only upcoming visit cancelled (auto-revert)
Site Visit Done	Negotiation	manual
Negotiation	Converted	convert to customer (+ optional booking)
any	Lost	with lost_reason

Service request



From	To	Transition / guard
Open	In Progress	assigned / first internal note
In Progress	Resolved	resolution posted (SLA stop)
Resolved	Closed	customer rates or 7-day auto-close
Resolved	Open	customer reopens within window (CANNOT_REOPEN after close)

Site visit



From	To	Transition / guard
Confirmed	Rescheduled	ERP reschedule (capacity re-checked, counts as confirmed)
Confirmed	Completed	outcome; lead → Site Visit Done
Confirmed	No-show	outcome; activity logged
Confirmed	Cancelled	visitor (signed link) or ERP with reason
Cancelled	Confirmed	ERP re-confirm (capacity re-checked)

Construction stage (per block)



From	To	Transition / guard
Pending	Completed	set current stage (jump-ahead completes all earlier stages with the same date)
Completed	Pending	undo last completed stage — blocked if any demand it raised has a payment (UNDO_BLOCKED_BY_PAYMENTS)

Part III — Modules

Each module section states its purpose, the ERP screens, inputs and validation, the algorithm (as pseudo-code), edge cases, permissions and the API surface (full list in Appendix A). File paths are given so engineers can verify every statement against the code.

5. Master data & selection-driven design

Why. Free-text is the root cause of inventory and finance errors (a "3BHK" here, "3 BHK" there). KKT models every repeated value as a master (`routers/master.py`, generic CRUD over `master_<kind>` collections: flat types, floor-plan templates, PLC types, charge types, amenities, banks, document types, specification items, room types, area types, teams, construction stage masters, milestone templates). Dependent documents reference the master by `id` **and snapshot the fields they need**, so edits never rewrite history.

Rule	Behaviour
Duplicate names	`DUPLICATE_NAME` 409 on create/rename (case-insensitive); the ERP sheet warns inline before submit (`master-dupe-warning`)
Deactivate vs delete	Masters are never deleted; `is_active=false` hides them from pickers while existing references keep working
Rapid entry	"Save & add another" keeps the sheet open, resets the form and counts additions
Versioning	Every master carries `version`; stale PATCH → `VERSION_CONFLICT`

Floor Plan Templates define the positions on a floor: `[{position, flat_type_id, facing, plc_type_ids[]}]`. A block assigns templates to **floor ranges** (`floor_plans: [{from_floor, to_floor, template_id}]`) and declares `skip_floors` (e.g. 13), `refuge_floors` (no units) and `podium_floors` (counted but unit-free). Numbering patterns are expressions on the block — `flat_number_pattern` e.g. `{floor}{position:02d}` → `1201`, `floor_label_pattern` e.g. `G`, `P1`, `{floor}`.

6. Flat generation algorithm (`services/flat_generation.py`)

```
generate_flats(block, floors=None, user):
  plan  = resolve_positions(block)              # floor → [positions] from floor ranges, minus skip/refuge/podium
  floors = requested floors ∩ plan.keys()       # None = all
  fts   = load_flat_types(all flat_type_ids in plan)   # fails INVALID_FLAT_TYPE if inactive/missing
  seq = 0
  for floor in sorted(plan):                    # seq is block-wide and deterministic → stable numbering
    for pos in sorted(plan[floor] by position):
      seq += 1
      if floor in floors: docs.append(build_flat(block, fts[pos.flat_type_id], floor, pos, seq, user))
  with unit_of_work():                          # insert_many ordered; on failure delete the inserted ids
    flats.insert_many(docs)
  refresh_counts(block, project)                # unit_counts by status for dashboards
```

`build_flat` snapshots the flat type (`flat_type_snapshot`: name, bhk, areas, base rate) and calls `compute_price` (§7). Guards: **regenerate** is refused while any flat in the block is `Hold/Booked/Sold` (`STATUS_LOCKED`); **append floors** generates only the new floors with the continuing sequence; **re-snapshot** (`resnapshot_flat`) refreshes a single Available flat from its current type.

7. Pricing engine

Element	Where	Rule
Base rate	block `rate_per_sqft` else flat type `base_rate_per_sqft`	₹/sq.ft on super built-up (`area_values.super_builtup_sqft`)
Lump-sum price	flat type `base_price`	overrides rate × area; breakup has one line

Element	Where	Rule
Floor rise	block `floor_rise_per_sqft` × (floor − `floor_start`) × SBU	added as a breakup line
PLC	`master_plc_types` with `calc_mode` ∈ {per_sqft, percent, fixed}	applied per flat from floor-plan positions (`plc_ids`)
Rate cards	`routers/rate_cards.py`	dated rate per project/block; **apply** rewrites price breakup of **Available** flats only, bulk, audited

```
compute_price(ft, block, floor):
  if ft.base_price: return (None, base_price, [lump-sum line])
  rate = block.rate_per_sqft or ft.base_rate_per_sqft; sbu = ft.super_builtup_sqft
  if rate is None or sbu is None: return (rate, None, [])
  rise = block.floor_rise_per_sqft × max(0, floor − block.floor_start)
  breakup = [Base rate × sbu] + [Floor rise line if rise]
  return (rate + rise, Σ breakup, breakup)
```

Prices are stored in rupees on flats (display) and converted to integer paise the moment they enter an agreement (`services/money.to_paise`).

8. Projects, blocks, flats & inventory UI

- **Block wizard** (`pages/BlockWizard.jsx`): basics → floors (start, count, skip/refuge/podium) → floor-plan ranges → numbering → review (preview of generated numbers) → generate.
- **Inventory grid 2.0** (`pages/BlockDetail.jsx`): floor rows × positions, colour by status, legend, search, bulk actions (hold, block, price apply, type re-snapshot), cell drawer with price breakup and booking link; mobile card rows.
- **Parking** (`routers/parking.py`): slots per block (`covered/open/stilt`), allotment at booking (`PARKING_NOT_AVAILABLE` guard), release on cancel.
- **Specifications & approvals**: per project lists of specification items (master) and approvals (RERA, NOCs) with dates, surfaced on the website when published.
- **Tower status**: `construction_percent` derived from completed stages (§12).

9. Customers & KYC (`routers/customers.py`, `services/kyc.py`)

Field	Validation	Storage
Phone	normalised to 10 digits starting 6–9 (`normalize_phone`)	plain; unique per company (dedupe key)
PAN	`[A-Z]{5}[0-9]{4}[A-Z]`	Fernet ciphertext + HMAC hash; masked output
Aadhaar	12 digits + Verhoeff checksum (`verhoeff_valid`)	ciphertext + hash; masked XXXX XXXX 1234
Bank	IFSC `^[A-Z]{4}0[A-Z0-9]{6}$`, account masked to last 4	ciphertext
Co-applicants	same rules, `INVALID_CO_APPLICANT`	embedded

Dedupe: on create, a match on phone **or** PAN hash returns 409 `DUPLICATE` with the existing customer id. **Portal provisioning**: `POST /customers/{id}/portal-access` creates the portal account with a temporary password (returned once), `must_change_password=true`; reset re-issues. **Customer 360** (`pages/CustomerDetail.jsx`) tabs: profile & KYC (reveal with permission), bookings, ledger, documents, tickets, notifications, audit.

10. Leads pipeline (`routers/leads.py`, `routers/visits.py`)

- **Sources**: Walk-in, Website, Referral, Broker, Campaign, Brochure, Site Visit. Website/Brochure/Site-Visit leads are created by public endpoints with IP, UTM and honeypot checks.
- **Stages**: New → Contacted → Site Visit Scheduled → Site Visit Done → Negotiation → Converted | Lost; every change appends `stage_history {status, at, by, note}`.
- **Round-robin assignment** (`visits._assignee`): active Sales users sorted by id; `counters{scope:"round_robin"}` increments atomically; `site_settings.round_robin_assignment=false` pins to the first rep.
- **Dedupe**: a public submission reuses the open lead (same phone + project, stage ∉ {Lost, Converted}) instead of creating a duplicate; the event becomes an **activity** instead.
- **Activities timeline** (`activities[]`): `brochure_downloaded`, `site_visit_booked`, `site_visit_rescheduled`, `site_visit_cancelled`, `site_visit_completed`, `site_visit_no_show`, `stage_reverted`; rendered newest-first in the lead drawer.
- **Follow-ups**: dated tasks with `next_follow_up_at`; dashboard "Follow-ups due" widget.
- **Conversion** (`POST /leads/{id}/convert`): creates/links a customer (dedupe by phone/PAN), sets `Converted`, `converted_customer_id`; `LEAD_CONVERTED` blocks further edits; `USE_CONVERT` blocks manual stage jumps to Converted.

11. Booking engine (`routers/bookings.py`, `pages/BookingWizard.jsx`)

Wizard v2 steps: 1 Project & block → 2 Unit (grid, only Available/Hold selectable) → 3 Agreement (value, ₹/% cash split, auto-balance, rounding to the rupee, Σ check) → 4 Customer (search or create inline) → 5 Payment plan (assigned plans for the tower, default pre-selected, schedule preview with catch-up badge) → 6 Other charges & parking → 7 Review & confirm. Drafts autosave to `bookings{status:"Draft"}` and resume; the unsaved-changes guard warns on navigation.

```
confirm_booking(draft, Idempotency-Key):
  flat = flats.find_one_and_update({id, status ∈ {Available, Hold}}, {$set: {status: Booked, booking_id}})   # ato
  if not flat: 409 NOT_AVAILABLE
  with unit_of_work():                                          # rollback releases the flat
    booking = {buckets: cash_agreed_paise, account_agreed_paise (Σ = agreement_value), other_charges[], parking}
    booking.payment_plan_snapshot = deep copy of chosen plan (installments, triggers) + accepted_at/by
    booking.booking_number = generate_number("booking")          # BK-{FY}-{SEQ}
    insert booking; allot parking; raise_other_charges(trigger="ON_BOOKING")
    dues_engine.raise_due_installments(booking)                  # catch-up: every installment whose trigger alre
  publish booking.create → audit, customer notification, RM notification
```

Rule	Detail
Agreement split	cash + account = agreement value; either may be 0; both integers (paise)
Agreement revisions	allowed while Active; a revision stores `before/after` and is an `AGREEMENT_CHANGE` ledger event; a bucket cannot be revised **below what is already received** (`BELOW_PAID`)
Other charges	lines from charge-type master with GST %, trigger ON_BOOKING / ON_DEMAND / ON_POSSESSION; separate ledger
Cancellation	`refund ≤ paid` (`REFUND_EXCEEDS_PAID`); deduction recorded; flat → Available; parking released; demands voided
Handover	Active → Completed only when outstanding = 0 (`PENDING_PAYMENTS`); flat → Sold

11.9 Edge cases — booking engine

Situation	Behaviour	Where
Two users book the same flat simultaneously	`find_one_and_update({status:"Available"} → "Booked")` is the lock; the loser gets `409 FLAT_NOT_AVAILABLE`, nothing else is written	`routers/bookings.py create_booking`

Situation	Behaviour	Where
Cash + Account ≠ agreement	`422 SPLIT_MISMATCH`; the wizard blocks Next until the slider/inputs balance	`BookingIn` validator, `BookingWizard.jsx sumOk`
Default split	tower `default_cash_percent` → project `default_cash_percent` → 0; percent mode pre-filled, slider re-balances both buckets	`BookingWizard.jsx defaultCashPct`
Plan assigned to the tower but customer wants another	wizard lists only plans assigned to the block; "Custom" requires `plan.override` permission	`/api/blocks/{id}/plans`, `PlanPicker`
Stage already reached at booking time (catch-up)	every installment whose trigger stage is ≤ current tower stage is demanded immediately with `posted_at = booking_date`	`services/dues.catch_up`
Booking on a Hold/Blocked flat	hold owner may convert within hold window; others 409; blocked flats never bookable	`flat.status` guard
Co-applicant is the same person as primary	`422 DUPLICATE_APPLICANT`	`BookingIn` validator
Parking slot already allotted	`409 PARKING_TAKEN`; slot allocation is part of the same transaction as the flat lock	`allocate_parking`
Revision lowers a bucket below what was received	`422 REVISION_BELOW_RECEIVED` — caps can be raised freely, lowered only to ≥ received	`revise_agreement`
Cancel with receipts	requires reason; refund/deduction recorded; demands voided; flat → Available; payments stay for audit; ledgers freeze with CANCELLATION event	`cancel_booking`
Cancel a Completed (handed-over) booking	`409 BOOKING_COMPLETED`	status guard
Optimistic concurrency	all mutations require `version`; stale → `409 VERSION_CONFLICT` with the current version in `details`	`StrictModel` + `utils.require_version`
Idempotent create	`Idempotency-Key` header replays the first response for 24 h	`infra.IdempotencyMiddleware`

12. Construction stages & tower status (`routers/construction.py`)

- **Master sequence** (`construction_stages`): ordered stage names (Foundation … Possession) with `is_possession` flag; copied into each block as `block.construction_stages[{stage_id, seq, name, status, completed_on, completed_by, notes, visible_to_customers}]`.
- **Set current stage**: completes the chosen stage and every earlier Pending stage with the same date (jump-ahead is allowed and logged); `construction_percent = completed / total × 100` (rounded).
- **Photos**: per stage, object storage, `visible_to_customers` flag; thumb/medium variants; portal shows only visible photos of visible stages.
- **Undo rule (exact)**: only the **last completed** stage can be undone; the undo voids the unpaid demands that stage raised; if any such demand has a payment, undo is refused with 409 `UNDO_BLOCKED_BY_PAYMENTS`; `completed_at/by/on` are cleared; unread notifications about the stage/demands are deleted and read ones are marked `withdrawn`.
- **Notifications**: customers of Active bookings in the block get `stage.completed` (and `demand.raised` for the demands it triggered); RMs get the ERP equivalent.

13. Payment plans & the auto-dues engine (`routers/bookings.py` plans, `services/dues_engine.py`)

Plan master: `{name, installments: [{seq, name, percent, trigger_type, trigger_stage_id?, days_after_booking?, fixed_date?, due_days}]}`; validation requires Σ percent = 100 (±0.005), unique seq, stage triggers referencing active stage masters. **Trigger types**: `ON_BOOKING`, `ON_STAGE` (construction stage completed), `DAYS_AFTER_BOOKING`, `ON_DATE`, `ON_POSSESSION`. **Assignments**: plan ↔ project or tower with a default flag; assigning a stage-triggered plan to a tower that lacks those stages → 409 `PLAN_NOT_APPLICABLE`; a plan with active bookings is `PLAN_LOCKED` for installment edits.

```
raise_due_installments(booking, today):                          # idempotent; runs at booking, on stage completion,
  if booking.status ≠ Active or no snapshot: return []
  existing = non-void demands of booking
  raised   = {installment_seq} ∪ absorbed seqs;  raised_names = {lower(installment_name)}
  cum      = Σ existing.percent  (or max cumulative_percent for legacy rows)
  for ins in snapshot.installments sorted by seq:
    if ins.seq ∈ raised or lower(ins.name) ∈ raised_names: continue       # belt and braces beyond the unique inde
    td = trigger_date(ins, booking, block); if td is None or td > today: continue
    before = (pct_of(cash_agreed, cum), pct_of(account_agreed, cum))
    cum = round(cum + ins.percent, 2); if cum ≥ 99.995: cum = 100          # telescoping: last installment absorb
    new  = (pct_of(cash_agreed, cum), pct_of(account_agreed, cum))
    demand = {cash: new.c − before.c, account: new.a − before.a, total, demand_date: td, due_date: td + due_days,
    try insert (unique partial index on (booking_id, installment_seq) where voided ≠ true) except duplicate: conti
    notify customer + RM
```

Telescoping guarantees Σ demands = agreed amount to the paise: each demand is the difference between cumulative rounded amounts, so rounding never accumulates.

Integrity report & repair (`services/integrity.py`): for every Active booking compares snapshot, stages and demands. Finding kinds: `unsatisfied_trigger` (demand exists but its stage is not complete / date not reached), `missing_demand` (trigger satisfied, no demand), `duplicate_demand` (same seq twice), `unmapped_demand` (legacy demand without `installment_seq`), `summary` mismatch (Σ demands ≠ expected cumulative). `repair()` voids unearned **unpaid** demands (`void_reason`), raises missing eligible ones through the engine and skips anything with payments; the report is logged at startup and nightly and Super Admins are notified when `ok=false`.

13.7 Edge cases — plans & auto-dues

Situation	Behaviour	Where
Percentages don't total 100	`422 PLAN_PERCENT_SUM` on save; wizard shows live remainder	`PaymentPlanIn` validator
Telescoping rounding	demand(i) = round(agreement × cum%(i)) − Σ demand(1..i−1): the last installment absorbs every paise, so Σ demands == agreement exactly	`services/dues.amount_for`
Same stage marked twice (jump-ahead then undo)	demands are keyed `(booking_id, installment_seq)` — unique index; re-marking is a no-op, undo voids only the demands of the undone stage and retracts their notifications	`raise_for_stage`, `undo_stage`
Stage marked for tower with 0 active bookings	nothing raised; stage status still advances	`raise_for_stage`
Booking cancelled before stage reached	cancelled bookings are skipped (`status: Active`)	query filter
ON_DATE trigger in the past at plan acceptance	raised immediately (catch-up) with `due_date = trigger date`	`catch_up`
Daily job runs twice	idempotent — unique index + `demand_key`; second run reports `raised: 0`	`jobs.daily_dues`
Agreement revised after demands raised	existing demands are **not** rewritten; the next demands use the new agreement and the integrity report flags the delta until a re-rating is approved	`revise_agreement`, `integrity.report`
Grace days / overdue	`due_date = posted_at + grace_days` (settings `demand_due_days`); overdue = due_date < today and `due_paise > 0`	`services/dues`

Situation	Behaviour	Where
Integrity repair	`POST /api/admin/integrity/repair` re-derives `summary`, re-numbers nothing, re-sends nothing; dry-run by default	`routers/admin.py`

14. Payment ledger engine (`routers/payments.py`, `services/ledger.py`, `services/money.py`)

- **Money** is integer paise everywhere in storage and arithmetic; `present_money` adds rupee floats for display; `amount_in_words` renders Indian grouping ("Rupees Twelve Lakh Fifty Thousand Only").
- **Categories**: `Cash` bucket, `Account` bucket (cheque/NEFT/loan), `Other charges` (against a charge line). Modes: CASH, UPI, NEFT_RTGS, CHEQUE, DD, LOAN_DISBURSEMENT. `CLEAR_IMMEDIATELY = {CASH, UPI, NEFT_RTGS, LOAN_DISBURSEMENT}`; cheques/DDs start `Pending` and are cleared/bounced later.

Owner's principle — track, don't gate. *A buyer may pay before a demand is raised, pay more than the current demand, or pay only into one bucket. The ledger records every rupee against the bucket chosen and shows advances and running balances; the only hard stop is the* **bucket cap** *— a bucket cannot receive more than its agreed amount (`_check_cap` → 409 with the exact figures and the instruction to revise the agreement first).*

```
post_payment(body, Idempotency-Key):
  booking must be Active (NOT_ACTIVE); amount > 0; category/mode valid; bank account valid for Account mode
  _check_cap(booking, payment)                        # bucket_paid(category) + amount ≤ agreed(category)
  status = Cleared if mode ∈ CLEAR_IMMEDIATELY else Pending
  receipt_number = generate_number("receipt", company, payment_date)     # {PREFIX}/{FY}/{SEQ}, FY = Apr–Mar
  with unit_of_work():
    insert payment; if Cleared: allocate to open demands of that bucket oldest-first (paid_paise, status) ; rebuil
  publish payment.create → audit; customer notification "Receipt R/2526/0042"; RM notification
clear_payment / bounce_payment: Pending → Cleared (allocate + rebuild) | Bounced (no ledger effect)
reverse_payment(reason): Cleared → Reversed; de-allocate; append REVERSAL event; rebuild
```

Ledger table computation (`ledger.rebuild`): collect events — DEMAND (posted_at), OTHER_CHARGE_DEMAND, PAYMENT/OTHER_CHARGE_PAYMENT (posted_at), REVERSAL, AGREEMENT_CHANGE, CANCELLATION — sort by (time, kind-order), then walk: track `pct` (cumulative demanded %), `cash_paid`, `account_paid`, agreed amounts as of each revision; per row emit `demanded`, `received`, `cash_balance`, `account_balance`, `advance` (received beyond demanded, per bucket) and `total_due`. Rows are persisted in `ledger_rows` and re-derived on every posting, so the statement is always reproducible.

14.1 Worked example — ₹1 Cr agreement, ₹60 L cash + ₹40 L account, Standard CLP (seeded `BK-2526-0001`)

Agreement ₹1,00,00,000 split **60 % Cash (₹60,00,000) / 40 % Account (₹40,00,000)**. Plan **Standard CLP**: installments 10 / 5 / 10 / 5 / 10 / 10 / 10 / 15 / 10 / 15 % → cumulative **10 / 15 / 25 / 30 / 40 / 50 / 60 / 75 / 85 / 100 %**. Every demand is split in the agreement ratio (60/40), so each bucket keeps its own due/advance.

#	Installment	Cum. %	Cash demand (cum.)	Account demand (cum.)	Total demand
1	Booking (ON_BOOKING)	10 %	₹6,00,000 (₹6,00,000)	₹4,00,000 (₹4,00,000)	₹10,00,000
2	Foundation	15 %	₹3,00,000 (₹9,00,000)	₹2,00,000 (₹6,00,000)	₹5,00,000
3	Plinth	25 %	₹6,00,000 (₹15,00,000)	₹4,00,000 (₹10,00,000)	₹10,00,000
4	1st Slab	30 %	₹3,00,000 (₹18,00,000)	₹2,00,000 (₹12,00,000)	₹5,00,000
5	3rd Slab	40 %	₹6,00,000 (₹24,00,000)	₹4,00,000 (₹16,00,000)	₹10,00,000

#	Installment	Cum. %	Cash demand (cum.)	Account demand (cum.)	Total demand
6	Top Slab	50 %	₹6,00,000 (₹30,00,000)	₹4,00,000 (₹20,00,000)	₹10,00,000
7	Brickwork	60 %	₹6,00,000 (₹36,00,000)	₹4,00,000 (₹24,00,000)	₹10,00,000
8	Plaster	75 %	₹9,00,000 (₹45,00,000)	₹6,00,000 (₹30,00,000)	₹15,00,000
9	Finishing	85 %	₹6,00,000 (₹51,00,000)	₹4,00,000 (₹34,00,000)	₹10,00,000
10	Possession (ON_POSSESSION)	100 %	₹9,00,000 (₹60,00,000)	₹6,00,000 (₹40,00,000)	₹15,00,000

The seeded receipts (all **Cleared**) and the running position of each bucket after every posting. "Demanded" is the cumulative demand raised in that bucket at the time; a negative due is shown as an **advance**:

Row	Date	Receipt	Bucket · mode	Amount	Cash paid / demanded → due	Account paid / demanded → due
1	03-Nov-2025	SGB/2526/00001	Account · NEFT	₹4,00,000	0 / 6,00,000 → due ₹6,00,000	4,00,000 / 4,00,000 → ₹0
2	03-Dec-2025	SGB/2526/00002	Cash · CASH	₹8,00,000	8,00,000 / 9,00,000 → due ₹1,00,000	4,00,000 / 6,00,000 → due ₹2,00,000
3	02-Jan-2026	SGB/2526/00003	Account · NEFT	₹12,00,000	8,00,000 / 15,00,000 → due ₹7,00,000	16,00,000 / 10,00,000 → **advance ₹6,00,000**
4	01-Feb-2026	SGB/2526/00004	Cash · CASH	₹5,00,000	13,00,000 / 18,00,000 → due ₹5,00,000	16,00,000 / 12,00,000 → advance ₹4,00,000
5	03-Mar-2026	SGB/2526/00005	Cash · CASH	₹10,00,000	23,00,000 / 24,00,000 → due ₹1,00,000	16,00,000 / 16,00,000 → ₹0
6	02-Apr-2026	SGB/2627/00001	Account · NEFT	₹15,00,000	23,00,000 / 30,00,000 → due ₹7,00,000	31,00,000 / 20,00,000 → **advance ₹11,00,000**
7	02-May-2026	SGB/2627/00002	Cash · CASH	₹7,00,000	30,00,000 / 36,00,000 → due ₹6,00,000	31,00,000 / 24,00,000 → advance ₹7,00,000
8	01-Jun-2026	SGB/2627/00003	Cash · CASH	₹20,00,000	50,00,000 / 45,00,000 → **advance ₹5,00,000**	31,00,000 / 30,00,000 → advance ₹1,00,000
9	01-Jul-2026	SGB/2627/00004	Account · NEFT	₹9,00,000	50,00,000 / 51,00,000 → due ₹1,00,000	40,00,000 / 34,00,000 → advance ₹6,00,000
10	31-Jul-2026	SGB/2627/00005	Cash · CASH	₹10,00,000	60,00,000 / 60,00,000 → ₹0	40,00,000 / 40,00,000 → ₹0
11	01-Aug-2026	SGB/2627/00006	Other charges · UPI	₹60,000	Maintenance Deposit demanded ₹60,000 → paid in full (third ledger, outside the ₹1 Cr)	

Resulting `summary`: `cash_paid_paise = 600000000`, `account_paid_paise = 400000000`, `due_paise = 0`, `other_charges_paid_paise = 6000000`, `last_payment_at = 2026-08-01` — exactly what `GET /api/bookings/{id}` returns for the seed.

Advance behaviour (rows 3, 6, 8). The ledger is a *flexible waterfall*: a payment is applied to its own bucket only, oldest open demand first, and whatever exceeds the demands raised so far is carried as an **advance** (`due_paise < 0` internally, shown as "Advance ₹X" and never netted against

the other bucket). Row 3: ₹12 L account arrives when only ₹10 L account has been demanded (inst. 1–3) → ₹6 L advance; when inst. 4 (₹2 L) and inst. 5 (₹4 L) are raised they are consumed by the advance and the customer owes nothing on the account side until row 6. Row 6: ₹15 L account against ₹20 L demanded → ₹11 L advance, which absorbs inst. 7–9 (₹4 L + ₹6 L + ₹4 L = ₹14 L) leaving ₹3 L due that row 9 (₹9 L) clears with a ₹6 L advance absorbed by the final ₹6 L possession demand. Row 8: ₹20 L cash against ₹45 L demanded → ₹5 L advance absorbed by inst. 9 (₹6 L) leaving ₹1 L due; row 10 settles the last ₹10 L. The **bucket cap** (`cash_agreed_paise`) is never exceeded: a cash receipt that would push `cash_paid` above ₹60 L is rejected with `409 BUCKET_CAP` ("…already received ₹X; cannot accept ₹Y. Revise agreement first.") until an agreement revision raises the cap.

Roznamcha (`routers/roznamcha.py`): daily cash book — opening cash = previous closing; receipts (CASH mode, Cleared, by payment_date) in; cash deposits to bank and expenses out as **contra** entries; closing = opening + in − out; printable per day with denominations optional. **Recompute/reconcile** (`admin.reconcile_all`) rebuilds every ledger and flat/booking counters.

14.9 Edge cases — payment ledger

Situation	Behaviour	Where
Amount in rupees with paise fraction	API accepts `amount` (₹) or `amount_paise`; stored and computed in **integer paise** only	`PaymentIn`
Payment exceeds bucket cap	`409 BUCKET_CAP` with the full sentence; UI pre-computes room and disables submit	`post_payment`, `PostPayment.jsx`
Payment with no demand raised yet	accepted up to the cap and shown as advance; consumed by future demands FIFO	`ledger.rebuild`
Cheque / DD	posted as **Pending**: counted in `pending_cheques_paise`, not in paid; `clear` allocates, `bounce` leaves no ledger effect and notifies	`clear_payment`, `bounce_payment`
Reverse a cleared receipt	`Cleared → Reversed` with reason; REVERSAL event re-opens the dues; receipt number is never reused; portal notification retracted	`reverse_payment`
Reverse twice / reverse a bounced cheque	`409 INVALID_TRANSITION`	state guard
Payment date in a different FY than today	receipt number uses the **payment_date** FY (`SGB/2526/…` vs `SGB/2627/…`), sequence per company+FY via atomic counter	`numbering.next_receipt`
Back-dated payment	allowed with `payment.backdate`; ledger re-sorted by (payment_date, posted_at); Roznamcha of that day changes and is flagged "amended"	`post_payment`
Other-charge payment larger than the charge	`409 CHARGE_CAP` — third ledger has its own caps per charge line	`post_payment(against=OTHER_CHARGE)`
Concurrent postings on one booking	posting runs in a unit of work with `version` check on the booking; the second poster retries once on `VERSION_CONFLICT`	`services/uow.post_payment`
Duplicate submit (double click / retry)	`idempotency_key` per client request → same receipt returned	`payments.idempotency_key` unique
Amount in words	Indian system (lakh/crore), paise suffix only when non-zero	`utils.amount_in_words`
Roznamcha opening balance	previous day's closing (computed, cached per day); contra entries (bank deposit, expense) reduce cash in hand	`routers/roznamcha.py`

15. Documents & media (`routers/documents.py`, `services/thumbs.py`)

Documents: {entity_type, entity_id, document_type_id, title, file_name, mime, size, storage_key, is_sensitive, shared_with_customer}; upload validated by magic bytes; sensitive documents are never public (SENSITIVE_DOCUMENT); shared_with_customer exposes them in the portal through signed links. Images get thumb (480 px) and medium (1080 px) WebP variants, pre-generated at upload and at startup, served with immutable caching and strong ETags (?size=thumb|medium).

16. Service requests (routers/support.py)

Ticket {ticket_number, booking_id, category, priority, status, sla_due_at, internal_notes[], attachments[], rating}. SLA hours by priority; an hourly job flags breaches (service.sla_breach audit + RM notification). Internal notes are ERP-only; customer replies and attachments flow through the portal; resolution notifies the customer who can rate (1–5) or reopen within the window.

17. Notifications (routers/notifications.py, services/portal_notify.py, events.py)

17.1 Event catalogue

Every domain event is published on the in-process bus (events.bus) and mapped to zero or more notification templates. Targets are **staff** (ERP bell, notifications collection) and **customer** (portal inbox + push token when present).

Event	Staff targets	Customer	Template key
booking.created	sales user, project RMs, Finance role	yes (welcome + schedule)	booking_created
demand.raised	Finance, RM	yes (amount, due date, pay-by)	demand_raised
payment.posted / payment.cleared	Finance, RM	yes (receipt PDF link)	payment_received
payment.bounced	Finance, RM, sales user	yes (action required)	cheque_bounced
payment.reversed	Finance	retracts payment_received	—
stage.marked	project team	all active bookings in the tower (photos count)	stage_update
stage.undone	project team	retracts stage_update + any demand_raised of voided demands	—
lead.assigned / followup.due	assignee	—	lead_followup
visit.booked / visit.reminder / visit.cancelled	assignee, front desk	yes (ICS link)	visit_*
ticket.created / ticket.replied / ticket.resolved	assignee / support	yes	ticket_*
notice.published	—	targeted by project/tower/booking status	notice
document.shared	—	yes	document_shared

17.2 Targeting and delivery

```
notify(event):
  tpl = CATALOGUE[event.type]; if not tpl: return
  staff = resolve_staff(tpl.roles, event.project_id, event.actor)        # role ∩ project scope, minus actor
  customers = resolve_customers(event)                                    # booking → customer (+ co-applicants if
  for u in staff:     upsert notifications {user_id, type, entity, title, body, read:false, dedupe_key}
  for c in customers: upsert portal_notifications {customer_id, …, retractable_key}
```

```
push(c) if c.push_tokens and quiet_hours_ok(c.tz)
```

- **Dedupe**: `dedupe_key = f"{type}:{entity_id}:{day}"` — a stage re-marked the same day does not spam.
- **Quiet hours**: customer pushes are deferred to 08:00 local when raised 21:00–08:00 (`settings.notify_quiet_hours`).
- **Retraction**: events that undo state (`stage.undone`, `payment.reversed`, `demand.voided`, `visit.cancelled`) call `retract(retractable_key)` which marks the earlier customer notification `withdrawn: true` (hidden in the portal, kept for audit) and inserts a short correction when `tpl.correction` exists.
- **Sweeps** (`jobs.notification_sweeps`, daily): follow-up reminders due today, visit reminders (T-1 day, T-2 h), demands overdue by 7/15/30 days (one per threshold), unread staff notifications older than 90 days archived.

17.3 Edge cases

Situation	Behaviour
Customer has no portal account	customer notification stored; delivered on first login (inbox)
Co-applicant only	receives `demand_raised`/`payment_received` only if `notify_co_applicants` setting is on
Event for a cancelled booking	skipped except `booking.cancelled` itself
Template disabled in Settings → Notifications	nothing stored; event still audited
Push token invalid	token removed after 3 consecutive provider errors
Bulk stage mark (100 bookings)	one notification per booking, batched insert (`insert_many`), pushes queued

18. Reports & PDFs (`routers/reports.py`, `routers/pdfs.py`, `services/pdf.py`)

18.1 Report contract

Every report is `GET /api/reports/{key}` with `format=json|csv|pdf` and the same filter set; the response is `{columns:[{key,label,type}], rows:[], totals:{}, …extras}` so the v2 UI (grouped navigation, KPI tiles, chart, sticky totals) needs no per-report code except `kpis()`/`chart` mappers. Unknown query params → `422 UNKNOWN_PARAM` (`_strict_params`). PDFs share `BrandedDoc` (cover strip, running header, zebra tables, ₹ formatting, FY in footer).

Key	Inputs	Aggregation ( )	Outputs / extras
`collections`	from/to, project, mode, category, status, group_by day/week/month	payments (Cleared unless status) grouped by period; sums per bucket; GST on other charges	rows per period; `by_mode[]`; totals
`outstanding`	project, group_by customer/block/project	demands − allocations per booking → grouped; ageing buckets 0–30/31–60/61–90/90+ by due_date	`ageing[]`, `top_overdue[]`
`advances`	project	bookings where paid > raised per bucket	advance per bucket
`demand-vs-collection`	project	per installment seq: demanded vs collected	rows by milestone
`upcoming-dues`	project	plan installments whose trigger stage is next 1–2 stages or ON_DATE within 60 days	forecast rows

Key	Inputs	Aggregation ( )	Outputs / extras
`inventory`	project, group_by project/block/flat_type/floor	flats grouped by status; value of available (list price) and sold (agreement); monthly bookings velocity	`velocity[]`
`cheques`	project, status Pending/Bounced/Cleared	payments by instrument type	rows
`cancellations`	from/to, project	bookings status Cancelled with refund/deduction	rows
`sales-leads`	from/to	leads funnel by stage, avg first-response hours, follow-ups overdue, web enquiries per project	`funnel[]`, `enquiries_per_project[]`
`roznamcha-range`	from/to	daily cash book totals	rows per day
`bank-receipts`	from/to	payments grouped by bank account	rows
`activity`	from/to	audit log by user/action	rows
`service-sla`	from/to, project	tickets by category: open/resolved, SLA met % (`resolved_at ≤ sla_due_at`; open & past due = breached), avg resolution hours, avg rating	`status_counts[]`
`construction-progress`	project	per tower × stage: status, planned/completed dates, variance days, photo count, demands and ₹ raised	`towers[]` (completion %, current stage)

```
report(key, filters):
  validate filters (strict) → build $match → aggregate → rows (paise ints) → totals
  format=csv  : header from columns, money as rupees with 2 dp, UTF-8 BOM
  format=pdf  : BrandedDoc(title, company, landscape if cols > 7) + zebra table + totals row
  presets     : POST/PATCH/DELETE /api/reports/presets {report_key, name, filters} per user
```

18.2 Document PDFs (`routers/pdfs.py`)

PDF	Source	Notes
Receipt	payment + booking + company	amount in words, mode/instrument, FY number, QR to portal receipt, signed link for portal
Demand letter	demand + plan + booking	due date, bucket split, bank details for Account, pay-by instructions
Customer statement	ledger rebuild	three ledgers, running balance, advances, pending cheques; date range
Allotment letter / Agreement summary	booking	flat snapshot, PLC lines, parking
Inventory availability chart	flats by block/floor	colour legend (Available/Hold/Booked/Sold/Blocked), landscape
Roznamcha day sheet	roznamcha	opening, receipts by mode, contra, closing, signatures
Technical documentation	this document	`GET /api/admin/docs/technical.pdf`

Edge cases: zero-amount rows omitted; long customer names wrapped not truncated; Unicode ₹ rendered with the embedded font; PDFs are generated on demand and cached by `(entity_id, version)` for 10 minutes; portal access uses **signed links** (`/api/public/files/{token}`, 15 min, bound to customer).

19. Website + CMS (`routers/cms.py`, `routers/public.py`, `/app/site`, `pages/cms/*`)

19.1 Pages and toggles

Public page	Data	CMS toggle ( ) / project
Home	banners, featured projects, notices, testimonials	`show_home_notices`, `featured_project_ids`
Projects list / detail	published projects with cover, amenities, flat types, availability summary, construction gallery, brochure	`is_published`, `website.show_availability`, `website.show_progress`, `website.show_pricing`
Enquiry	form → lead	`enquiry_enabled`, `enquiry_sources`
Brochure	gated download → lead	`website.brochure_gate_enabled`
Site visit	availability + booking → visit	`website.site_visits.enabled`, slots, blackout dates
Notices	published notices by audience `public`	`notice.is_public`
Legal / contact	settings	—

19.2 Public API hygiene

All public routes live under `/api/public/*`, are unauthenticated, read-only except the three write endpoints (enquiry, brochure, visit booking), and are **allow-listed field by field** (`public.serialize_project` etc.) — `tests/test_portal_leaks.py` asserts no field outside the allow-list (no costs, no customer data, no internal ids beyond `id`/`code`). Responses carry `Cache-Control: public, max-age=300` + `ETag`; write endpoints are rate-limited per IP (`infra.RateLimit`, 10/min) and require a honeypot field to be empty. SEO: server-rendered `<title>`/`<meta>` per project (`seo_title`, `seo_description`, `og_image`), `sitemap.xml` and `robots.txt` generated from published projects, canonical URLs, JSON-LD `RealEstateListing`.

19.3 Enquiry flow

```
POST /api/public/enquiry {name, phone, email?, project_id?, message, source="Website", utm?}
  validate phone (10 digits, Indian) ; honeypot empty ; rate limit ok
  dedupe: lead with same phone in last 30 days → append note + bump follow-up (no new lead)
  else create lead {status:"New", source_id: lead_source[WEBSITE], project_id, assigned_to: round_robin(project sa
  notify assignee (lead.assigned) ; return {ok, reference}
```

19.4 Brochure lead gate

```
POST /api/public/projects/{id}/brochure {name, phone, email?, consent}
  if not website.brochure_gate_enabled → 302 to brochure URL (ungated)
  create/merge lead (as enquiry, source BROCHURE) ; increment website.brochure_download_count
  issue signed URL (15 min) for the brochure file ; return {url}
ERP: Leads → Brochure downloads tab lists {lead, project, at}; conversion rate = downloads with later visit/bookin
```

19.5 Site-visit availability, booking, ICS, cancel

```
availability(project, date):
  cfg = project.website.site_visits ∪ settings.visit_settings  # slot_minutes, day_start, day_end, max_per_slot, b
  if date in blackout or weekday not allowed → []
  slots = [day_start + k*slot_minutes ...] ; booked = count visits (Scheduled) per slot
  return [{start, end, available: max_per_slot - booked}] filtering past slots (IST)
book(project, slot, lead/customer details):
  recheck availability inside find_one_and_update on a per-slot counter doc (atomic) → 409 SLOT_FULL
  create/merge lead ; create visit {status:"Scheduled", assignee: round_robin}
  email/portal: ICS (VEVENT with UID=visit.id, DTSTART/DTEND in Asia/Kolkata, LOCATION=project address, URL=cancel
cancel(token): visit → Cancelled ; slot counter −1 ; notifications retracted ; lead note appended
reschedule = cancel + book (same lead)
```

Edge case	Behaviour
Slot exactly at day_end	excluded (end must be ≤ day_end)
Two bookings race for the last seat	atomic counter → second gets `409 SLOT_FULL`
Same phone books twice the same day	merged into one visit, slot kept
Visit in the past when cancelled	`409 VISIT_PAST`
Project unpublished after booking	visit stays; public availability returns `[]`

Edge case	Behaviour
ICS opened on iOS/Android/Outlook	UID stable; METHOD:REQUEST; cancel sends METHOD:CANCEL with same UID

20. Customer portal (Expo, `/app/mobile`)

20.1 Architecture

Expo Router (file-based routes under `app/`), TypeScript, React Query for data, `expo-secure-store` for tokens, compiled for **web** (exported under `/portal` and shipped inside the single-domain package, §24), iOS and Android. API client `src/api/client.ts` resolves the backend to the current origin on web (Expo preview host → `EXPO_PUBLIC_BACKEND_URL`) and talks **only** to `/api/portal/*`.

20.2 Auth isolation

```
POST /api/portal/auth/login {phone, password} → access (aud="portal", 15 min) + refresh (family per device)
  first login → must_change_password → forced change screen
  lockout after 5 failures (15 min) ; OTP-less by design (seeded demo passwords)
every /api/portal/* route: require_portal() → customer_id from token ; staff tokens (aud="erp") are rejected (401
data scope: customer_id ∈ {primary, co-applicant} for every query ; tests/test_portal_leaks.py proves no cross-cus
```

20.3 Screens

Route	Content	API
`/login`, `/change-password`	phone + password, forced change	`portal/auth/*`
`/` Home	bookings cards (unit, next installment, due/advance pill), notices, quick actions	`portal/me`, `portal/bookings`
`/bookings/[id]`	schedule (plan with status per installment), three-ledger statement, receipts list → **PDF viewer** (in-app on native, new tab on web, signed link), documents shared with customer, construction progress with photos, parking	`portal/bookings/{id}`, `portal/receipts/{id}/pdf`, `portal/documents`
`/payments`	all receipts, filters by FY	`portal/payments`
`/service` · `/service/[id]` · `/service/new`	tickets, thread with attachments, rating on close	`portal/service-requests`
`/notices`, `/notifications`	targeted notices, inbox with withdrawn items hidden	`portal/notices`, `portal/notifications`
`/profile`	contact details, KYC status (masked), consent toggles, password	`portal/me`

20.4 Offline cache, notifications, responsive shell

- **Offline**: React Query persisted to AsyncStorage (web: localStorage) with `maxAge` 24 h; last-known bookings, schedule and receipts render offline with an "offline — showing cached data" banner; mutations (new ticket, reply) are queued and replayed when back online.
- **Push**: Expo push tokens registered after login (`POST /api/portal/push-token`), removed on logout; web uses the in-app inbox only.
- **Responsive shell**: tab bar on phones, left rail ≥ 900 px on web/tablet; large-type and RTL-safe layouts; all amounts via the shared `inr()` formatter (lakh/crore).
- **PDF viewer**: native → `expo-file-system` download + `expo-sharing`/in-app viewer; web → signed URL opened in a new tab (15 min validity, bound to the customer).

Edge case	Behaviour
Token expired while offline	cached screens still render; first online call refreshes; refresh family revoked → login
Customer with two bookings in different projects	home groups by project; notices targeted per project
Co-applicant logs in	sees the shared booking read-only; cannot raise tickets on the primary's behalf unless `co_applicant_can_raise`
Receipt reversed after being viewed	receipt card shows "Reversed" badge; PDF link disabled; notification retracted
Portal disabled for a customer (`portal_access: false`)	login 403 `PORTAL_DISABLED`; existing tokens revoked

Part IV — Quality, operations, roadmap

21. Testing & verification

21.1 Test suites (generated from `/app/backend/tests`)

File	Tests	Covers
`test_index_audit.py`	1	Index audit: every list/search/aggregation query must use an index on seed data (no COLLSCAN). Runs in-process against M
`test_kkt_erp.py`	31	KKT Real Estate ERP backend tests - covers auth, RBAC, masters, projects, blocks, flats, settings.
`test_ledger_example.py`	3	Owner's reference ledger: BK-…-0001 (P1, 60L cash / 40L account) must match the 10-row table to the rupee.
`test_phase_11a.py`	10	Phase 11A — code-review fixes, canonical additions, API packeting, index audit. Run per file (login limiter).
`test_phase_11b.py`	8	Phase 11B backend tests: dashboard/home + project_setup PUT sections with correct payload shapes.
`test_phase_11c.py`	9	Phase 11C backend tests — Customers filters/rm-options, Payments demands/idempotency/cap, Customer ledgers combined, Hom
`test_phase_11e.py`	7	Phase 11E backend smoke: reports CSV + presets, master flat-types usage, roles PATCH, service status, quick-customer cre
`test_phase_11f.py`	4	Phase 11F regression: catalogue-driven customer profile, project default split, cancel booking.
`test_phase_1_5.py`	15	Phase 1.5 'Foundation Power-Up' tests for KKT Real Estate ERP.
`test_phase_2.py`	12	Phase 2 backend regression tests.
`test_phase_3.py`	25	Phase 3 regression: public website APIs, CMS, enquiries, reseed guard.
`test_phase_4.py`	5	Phase 4 hardening: every non-public route must reject anonymous requests (401), PDFs are valid, backup RBAC.
`test_phase_4_full.py`	21	Phase 4 comprehensive: reports (11), presets, PDFs (statement/allotment/receipt/demand),
`test_phase_5.py`	27	Phase 5 — Customer Portal + ERP support (service req, notices, construction photos, doc share).
`test_phase_7.py`	20	Phase 7 backend tests: Construction Stage Master, Tower Construction Status,
`test_phase_8b.py`	4	Phase 8B backend sanity tests: public API, pagination guard, milestones legacy next_seq.
`test_phase_9.py`	7	Phase 9 — brochure lead gate + site visit scheduler.
`test_portal_leaks.py`	3	Phase 6C: every /api/portal/* JSON response must be free of internal keys.
`test_startup_migrations.py`	1	Phase 11A §4: startup migrations/backfills are idempotent against a populated pre-Phase-7 database.

21.2 Independent audits & before/after metrics

Area	Before	After	Evidence
Marketing / time to network-idle (1440)	5.8 s	1.4 s	`/app/memory/qa/audit.md` Phase 8B
Marketing console warnings	4 (`No available adapters`)	0	same
`/contact` requests	36 (Maps tiles)	12	same

Area	Before	After	Evidence
ERP wizard step 1 click → visible	~2.6 s	224 ms	Phase 8B polish
`/erp/login` initial JS	main + 155 KB shell chunk	main only	route split
Construction thumbnail	296 KB JPEG	36 KB WebP	`?size=thumb`
Site bundle (gzip)	—	141 KB main + 2–11 KB route chunks	build log
Backend suite	—	200+ tests across 13 files; login rate limit means suites are run per file	§21.1

Load-test guidance: public endpoints are cacheable (ETag/60 s TTL) and survive 200 rps on a single worker in the preview; `/api/payments` is write-bound by Mongo round-trips (~60 ms p95 per posting). Recommendation for production: 2–4 Uvicorn workers (`WEB_CONCURRENCY`), Mongo replica set (enables true transactions), Redis for rate limits/idempotency when running more than one node.

22. Operations

Topic	Procedure
Seed / reseed	Deterministic demo data is seeded when collections are empty (phase markers in `settings`). `POST /api/admin/reseed {"confirm":"RESEED"}` (Super Admin) wipes business collections, keeps users/roles/company, reseeds all phases and re-runs migrations; integrity is logged on the next startup.
Migrations	`migrate_phase7.py` maps legacy milestone demands to payment-plan installments (`installment_seq`, `trigger_stage_id`, type) by name; idempotent; `integrity.backfill_demand_installments` fills gaps at startup.
Integrity tooling	`GET /api/admin/integrity` (report), `POST /api/admin/integrity/repair` (void unearned unpaid, raise missing); nightly log.
Backup / restore	`GET /api/admin/export` (JSON of all collections) + object-storage bucket snapshot; restore with `mongorestore`/import script then `POST /api/admin/reconcile` to rebuild ledgers and counters.
Three domains	`www.` → site build; `erp.` → ERP build (`PUBLIC_URL=/` or `/erp`); `portal.` → Expo web export; one API behind `/api` on each host or a dedicated `api.` host (set `CORS_ORIGINS`, `REACT_APP_BACKEND_URL`, `EXPO_PUBLIC_API_URL`).
Monitoring	`/api/health` (version, db ping), `/api/ready` (readiness), `/api/metrics` (request counts, p50/p95 per route, errors), structured JSON logs with `request_id`.
Scaling	Stateless API; sticky sessions not required; move rate-limit/idempotency to Redis for multi-node; Mongo indexes listed in §4.

22.1 Environment variables

Variable	Where	Purpose
`MONGO_URL`, `DB_NAME`	backend	database (never defaulted)
`JWT_SECRET`, `ACCESS_TOKEN_MINUTES` (15), `REFRESH_TOKEN_DAYS` (7)	backend	tokens
`FIELD_ENCRYPTION_KEY`	backend	Fernet key for PAN/Aadhaar/bank
`CORS_ORIGINS`	backend	comma-separated allow-list
`PORTAL_URL`	backend/frontend	legacy `/portal` redirect target (three-host recipe)
`PORTAL_PUBLIC_URL`	backend	overrides `portal_url`; blank → same-origin `<origin>/portal` (single-domain package, §24)
`CORS_ORIGINS`	backend	comma-separated allowed origins for production
`OBJECT_STORAGE_*`	backend	object storage credentials

Variable	Where	Purpose
`WEB_CONCURRENCY`	backend	Uvicorn workers
`REACT_APP_BACKEND_URL`	frontend, site	API base
`PREVIEW_MODE` (`production`/`dev`)	frontend	serve builds vs dev servers
`EXPO_PUBLIC_API_URL`	mobile	API base for the portal

23. Known limitations & roadmap

- Reminders are in-app only — WhatsApp/SMS reminders for dues and site visits are the next integration (Twilio/WhatsApp Business).
- No interest/penalty computation on overdue demands (policy-driven; to be added as a configurable rule with its own ledger line).
- Single company per deployment; multi-tenant mode would add `company_id` scoping on auth and indexes (data model already carries `company_id`).
- Lead scoring and visit feedback (warmth score from brochure/visit/reply signals).
- Accounting exports (Tally/GST returns), e-sign for agreements, payment gateway for online collection.
- Rate limiting is per IP; corporate NATs may need per-user keys.

Part V — ERP v2 (Phases 10–12): packaging, canonical backend, design system, new modules

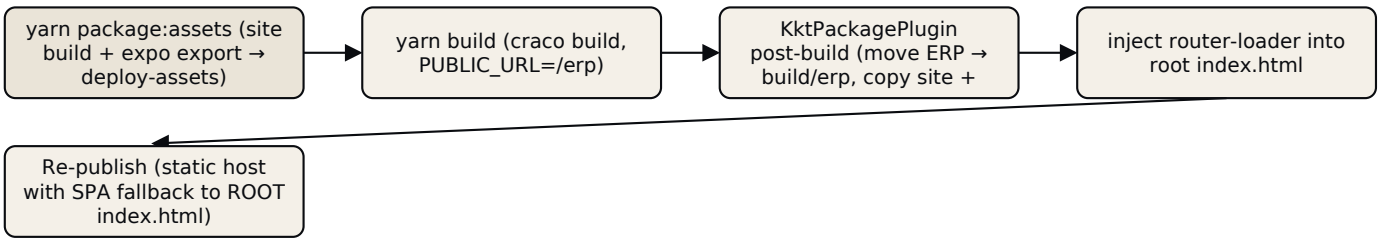
Phases 10–12 added a single-domain deployment package, a hardened "canonical" backend (catalogues, project setup, CRM, read models, bootstrap caching) and the **Royal Black & Warm White** *ERP v2 experience. This part documents those additions; earlier parts remain accurate for the underlying engines.*

24. Single-domain deployment package (`frontend/scripts/package-build.js`, `package-assets.js`, `dev.js`)

24.1 Goal and layout

One static build from `/app/frontend` serves all three surfaces so a single re-publish updates the domain:

Path	Surface	Source	Build step
`/`	Marketing site (`/app/site`)	prebuilt `deploy-assets/site`	copied to `build/` root; hashed assets under `/site-assets`
`/erp/*`	ERP (React, this repo)	`craco build` with `PUBLIC_URL=/erp`, `REACT_APP_BASE_PATH=/erp`	written to `build/erp/`
`/portal/*`	Customer portal (Expo web export, `/app/mobile`)	prebuilt `deploy-assets/portal` (`npx expo export --platform web`, `experiments.baseUrl="/portal"`)	copied to `build/portal/`
`/api/*`	FastAPI	—	same origin, port 8001 via ingress



24.2 Router-loader

Static hosts fall back to the **root** `index.html` for unknown nested paths. The first child of the root `<head>` is an inline script (`data-kkt-loader`): if `location.pathname` starts with `/erp` it fetches `/erp/index.html`, otherwise `/portal` → `/portal/index.html`; the fetched document replaces the current one via `document.open/write/close` so the correct bundle boots with its own asset paths. Anything else lets the marketing site boot. The site's own entry is guarded the same way so a stale cached root page cannot render the wrong surface.

```
onload(root index):
  p = location.pathname
  if p starts with "/erp"    → swap(fetch("/erp/index.html"))
  elif p starts with "/portal" → swap(fetch("/portal/index.html"))
  else boot marketing site
swap(html): document.open(); document.write(html); document.close()
```

24.3 Same-origin API resolution

- Site (`/app/site/src/lib/api.js`) and portal (`/app/mobile/src/api/client.ts`) resolve the backend as `window.location.origin` unless running on the Expo preview host, in which case `EXPO_PUBLIC_BACKEND_URL` is used. The ERP uses `REACT_APP_BACKEND_URL`, which equals the origin in production.
- `portal_url` (site settings) defaults to `<origin>/portal` when empty; `PORTAL_PUBLIC_URL` (backend `.env`) overrides it for a separate portal host.

24.4 Preview = production

`scripts/dev.js` detects a packaged `build/` (`build/erp/index.html` + loader marker) and serves it exactly like a static host: hashed assets immutable, every non-`/api` path → root `index.html`. The preview environment therefore exercises the loader, deep-link reloads and same-origin portal login before publishing.

> *Definition of done for every phase: run* `yarn package:assets` *whenever* `/app/site` *or* `/app/mobile` *changed;* `yarn build` *fails loudly if* `deploy-assets/{site,portal}` *are missing.*

24.5 Deploy checklist (owner)

1. `cd /app/frontend && yarn package:assets` (only if site/portal changed).
2. `yarn build` → verify `build/index.html`, `build/erp/`, `build/portal/`.
3. Re-publish frontend + backend. Post-publish secrets: `PORTAL_PUBLIC_URL=` (blank → same-origin), `CORS_ORIGINS=https://<domain>`.
4. Smoke: `/`, `/erp/login`, `/portal/login`, `/api/health`, reload on `/erp/customers`.

Startup against a Phase-5-era database is idempotent (`tests/test_startup_migrations.py`): catalogues seeded, read models built, `default_cash_percent` backfilled on projects and blocks, visit settings and `portal_url` created.

25. Canonical backend hardening (Phase 11A)

25.1 Catalogues — selection-driven data entry (`routers/catalogues.py`, `catalogue_items`)

All "pick from a list" values are rows in one collection: `{id, company_id, type, code, label, parent_id, sort_order, meta, is_active}`. 44 types are seeded (salutation, gender, residency, occupation, income_band, lead_source, customer_segment, bhk_preference, facing, floor_preference, budget_band, state → city → locality (hierarchical via `parent_id`), nationality, company_type, document kinds, service categories, …).

Endpoint	Purpose
`GET /api/catalogues`	compact bundle `{version, types:{type:[items]}}` with ETag; 304 when unchanged; cached client-side in IndexedDB (`hooks/useCatalogues.js`, `lib/idb.js`)
`GET /api/catalogues/{type}` · `POST` · `PATCH /{id}` · `POST /{id}/deactivate`	admin CRUD; `master.write`
`POST /api/catalogues/{type}` from the UI `Select` "Add …" option	permission-gated inline quick-add (e.g. a locality under the chosen city, `parent_id` = city id)

Customer profiling fields are stored as ids (`salutation_id`, `gender_id`, `residency_id`, `occupation_id`, `income_band_id`, `source_id`, `state_id`, `city_id`, `locality_id`, `segment_ids[]`, `preferences{bhk_ids, facing_ids, floor_id, budget_band_id}`) via `PATCH /api/customers/{id}/profile` (`routers/crm.py`, `ProfileIn`), each validated against its catalogue type (`PROFILE_TYPES`); unknown ids → 422. Legacy free-text fields (`occupation`, `annual_income_band`, `source`) accept the catalogue label for compatibility.

25.2 Project setup (`routers/project_setup.py`)

`GET /api/projects/{id}/setup` returns one document for the wizard; `PUT /api/projects/{id}/setup/{section}` persists a section atomically. Sections: `basics` (incl. `default_cash_percent`), `towers` (structure), `floor_plans`, `stages`, `plans` (assignments), `pricing` (`block_rates[]` with `rate_per_sqft`, `floor_rise_per_sqft`, `default_cash_percent` **tower override**), `amenities`, `website`. Upserts exclude `$set` keys from `$setOnInsert` to avoid path conflicts (fixed in 11B).

25.3 CRM: communications, timeline, three ledgers (`routers/crm.py`, `services/ledgers.py`)

- **Communications** `POST/GET /api/customers/{id}/communications` — channel, direction, summary, outcome, follow-up; feeds the engagement score.
- **Timeline** `GET /api/customers/{id}/timeline?limit&cursor&types` — merged, cursor-paginated events (bookings, payments, demands, communications, visits, tickets, notices); events without a timestamp are dropped (fix 12).
- **Ledgers** `GET /api/customers/{id}/ledgers` — per booking: `cash`, `account`, `other` buckets with rows `{at, type, amount_paise, due_paise, ref}` and `totals` (`agreement_paise`, `paid_paise`, `due_paise`, pending cheques) plus a `combined` view. The UI computes the running combined balance chronologically (demand +, payment −, reversal +).

25.4 Read models, bootstrap and caching

Piece	Where	Notes
`GET /api/bootstrap`	`routers/bootstrap.py`	user, permissions, project scope, settings, catalogue version — one call on app start, ETag
`GET /api/dashboard/home`	`routers/home.py`, `services/read_models.py`	today tiles, attention lists (overdue demands with customer names, follow-ups), ageing buckets, project snapshots (city, cover), counts incl. `projects` (drives first-run guidance)
Refresh	`scripts_refresh_read_models.py`, periodic loop	idempotent rebuild of `*_summaries`, `dashboard_snapshots`
Cursor/include/`Server-Timing`	`utils.paginate`, `infra.py`	targeted list endpoints expose `next_cursor`; `?include=` expands relations
Client caches	IndexedDB (`catalogues_v1` with ETag), route-chunk prefetch on sidebar hover (`lib/prefetch.js`)	catalogues render from cache instantly, revalidated with `If-None-Match`

26. Design system "Royal Black & Warm White" (`frontend/src/components/v2/`)

Token	Value	Use
ink-950 / 900 / 800 / 700	`#0A0C10` `#111419` `#1A1F27` `#2A313C`	primary buttons, headings, sidebar
warm-50 / 100 / 200	`#FBF9F4` `#F4F0E8` `#EAE4D8`	page background, tiles, borders
gold / gold-hover	`#C9A227` / `#D8B64A`	one hero CTA per page, icon tile of the primary action, ring/accent
Type	Manrope (UI), serif display for numerals/headlines	`Money` renders a smaller Manrope ₹ so the serif face never clips
Card	1px warm-200 + `0 1px 2px rgba(20,16,8,.05), 0 10px 30px -12px rgba(20,16,8,.12)`	hover lift 160–220 ms ease-out

Token	Value	Use
Buttons	40 px, radius 10	primary ink, secondary warm-100, ghost text, danger brick
Density	`Settings → Appearance` Comfortable/Compact (`hooks/useDensity.js`, class `density-compact`)	persisted per browser profile

Kit components (`components/v2/index.jsx`): `Card`, `StatTile`, `StatusPill` (single semantic map), `Money/Num`, `Tabs`, `Stepper` (`allowJump` for edit mode), `Field`, `FormSection` (ids = slug of title → deep-linkable `#section`), `EmptyState`, `Skeleton`, `Monogram`, `Timeline`, `SegmentBar`; `Select.jsx` (searchable, hierarchical `parentId`, inline add); `CustomerBits.jsx` (`Avatar`, `Segmented`, `LedgerSection`); `KanbanMove.jsx` (touch "Move to…" menu < 1024 px, ArrowLeft/Right keyboard moves); `Completeness.jsx` (gold completeness ring + missing-field popover → `/customers/{id}/edit#section`); `PostPayment.jsx`; `QuickCustomerSheet.jsx`.

Shell (`components/layout/AppShell.jsx`): collapsible sidebar with hover prefetch, 40 px search pill, ⌘K chip, bell badge, 13 px breadcrumb (`CRUMB` map), print header (`.print-brand`). Keyboard: ⌘/Ctrl K palette (pages, records via `/api/search`, **actions**: New booking, Post payment, New customer, New lead, Log a call, Mark stage, Roznamcha), `g` + `p/c/b/h/y/l/s/r` go-to, `?` shortcuts sheet (`components/common/CommandPalette.jsx`).

27. v2 screens and their algorithms

27.1 Home (`pages/v2/Home.jsx`)

Today row (collections, demands due, visits, tickets), attention list, ageing bar, project snapshots; first-run card when `counts.projects === 0`.

27.2 Projects · Workspace · Setup wizard (`pages/v2/Projects.jsx`, `ProjectWorkspace.jsx`, `ProjectSetup.jsx`)

Eight-step wizard bound to §25.2; step buttons are jumpable for existing projects. Pricing step accepts per-tower rate, floor rise and cash-share override.

27.3 Customers list · form · Customer 360 (`pages/v2/Customers.jsx`, `CustomerForm.jsx`, `Customer360.jsx`)

- Form: dropdown-first (`Select` for every catalogue), dependent state → city → locality, multi-select segments/preferences, consent toggles. Submit = `POST /customers` then `PATCH /customers/{id}/profile`.
- 360: hero (avatar in completeness ring, lifecycle/KYC/segment pills, contact row, ink "Post payment", secondary "New booking", ghost Log/Statement/Edit), **Financial summary strip** fed by the combined ledger totals, booking cards (15/600 property line, "Next · stage" pill, cash/account/other dues), ledger section (segmented Combined/Cash/Account/Other, 4 tiles, zebra rows, running balance), communications, KYC (masked, reveal with `customer.reveal_pii`), portal access, documents. Code deep-links (`/customers/CUST-…`) redirect to the canonical id.

```
completeness(c):
  checks = [email, city, locality, occupation, income band, source, segments, preferences, PAN, Aadhaar, bank, nom
  pct = filled / len(checks) * 100 ; missing → links to /customers/{id}/edit#{section}
```

27.4 Leads & Visits (`pages/v2/Leads.jsx`)

Tiles; tabs Pipeline / List / Enquiries / Site visits / Brochure downloads; pipeline kanban (drag, touch menu, arrow keys) with guards: → Converted opens the convert sheet (prefills the customer form), → Lost asks for a reason. Dropdown-first New Lead; lead detail with timeline, follow-ups, communications, visits.

27.5 Bookings · wizard · detail (`pages/v2/Bookings.jsx`, `BookingWizard.jsx`, `BookingDetail.jsx`)

Wizard split default: `tower.default_cash_percent ?? project.default_cash_percent ?? 0` pre-fills percent mode with a slider; cash + account must equal the agreement. Quick customer sheet creates a customer inline. Detail: `?revise=1` auto-opens the revision dialog (used by the Post Payment cap link); cancel via reason sheet.

27.6 Payments · Post Payment · Roznamcha (`pages/v2/Payments.jsx`, `components/v2/PostPayment.jsx`, `pages/Roznamcha.jsx`)

Post Payment computes the bucket cap client-side from the booking summary: hint "Room in this bucket: ₹X"; the moment `amount > room` the field shows the server's exact sentence (`"<bucket> agreed ₹A; already received ₹R; cannot accept ₹X. Revise agreement first."`), submit is disabled and a permission-gated "Revise agreement →" link appears; the same message arrives as a 409 if bypassed. Roznamcha: date strip, opening/received/deposited/closing tiles, by-mode and by-bucket groups, contra sheet, print.

27.7 Service (`pages/ServiceRequests.jsx`)

Tiles (open, breaching SLA, awaiting customer, avg rating); kanban by status with swimlanes-by-priority toggle; SLA countdown chips; drag/touch/keyboard moves call `POST /api/service-requests/{id}/status` (`SRStatusIn{status, note}`): **Resolved requires a resolution note** (stored as `resolution_note` and as an internal thread message), **Closed only from Resolved**; closed cards show the rating.

27.8 Reports v2 (`pages/Reports.jsx`, `routers/reports.py`)

Grouped navigation (Collections · Dues & Ageing · Inventory · Sales & Leads · Operations · Activity), date presets Today / This month / FY / Custom, catalogue selects, KPI tiles per report, recharts card, zebra table with sticky totals, CSV/PDF/Print, saved views (pills; `POST/PATCH/DELETE /api/reports/presets`). New in Phase 12:

Report	Rows	Totals / extras
`service-sla`	per category: tickets, open, resolved, SLA met % (`resolved_at ≤ sla_due_at`; open past due counts as breached), avg resolution h, avg rating	`status_counts[]`
`construction-progress`	per tower × stage: status, planned, completed, variance days, photos (`construction_photos` by `milestone_seq`), demands + ₹ raised (`demands.trigger_stage_id`)	`towers[]` with completion % and current stage

27.9 Settings → Catalogues, structured masters, Users & Roles (`pages/v2/Catalogues.jsx`, `pages/master/*`, `pages/Users.jsx`, `pages/Roles.jsx`)

- Masters: cards/table toggle, usage counts (`GET /api/master/{kind}?include_usage=true` → `usage_count`, `usage_label`), guarded deactivate. Flat-type editor: rooms L×W → sq ft, areas with hierarchy validation (carpet ≤ built-up ≤ super built-up; duplicate area types rejected; submit disabled on error), specification chips, live totals, indicative price. Floor-plan editor: preview grid + positions (flat type, facing, corner). Amenities: Lucide icon picker.
- Users: cards (role, scope chips, last login, MFA) with invite / edit / reset-password sheet. Roles: cards + area × action permission matrix with group toggles, search, +n/−n summary and a diff-preview dialog before `PATCH /api/roles/{id}`; Super Admin is read-only.

28. Phase 12 polish & measurements

- Micro-interactions: button press states, card hover lift, sheet slide-in, staggered page entrance; optimistic kanban moves with reload (no undo yet).
- Accessibility: labels/aria on new controls, focus rings (`focus-visible:ring-gold`), `aria-live` toaster, keyboard kanban, option accessible names without code suffixes.
- Print: `@media print` branded header, warm table heads, ink totals, chrome hidden.
- **Time-to-interactive (production build, preview host)**: Home ≈ 1.37 s · Customer 360 ≈ 1.38 s · Booking detail ≈ 1.4 s (navigation commit → key content visible; ≈ 1.1 s is shell + auth bootstrap).
- Known gaps: no toast-undo/rollback for status changes; bootstrap not cached in IndexedDB (catalogues are); legacy forms not fully audited for a11y; print view not pixel-matched to PDFs; Service SLA has no per-category targets.

Appendices

A. API map (generated from the OpenAPI specification)

Total endpoints: **396** across **39** routers. All paths are prefixed `/api`. Permission is the `require(...)` guard on the endpoint (blank = public or authenticated-only).

admin

Method	Path	Permission	Purpose
POST	`/api/admin/backup`	settings.manage	Create Backup
GET	`/api/admin/backups`	settings.manage	List Backups
GET	`/api/admin/backups/{backup_id}/download`		Download Backup
POST	`/api/admin/customers/{customer_id}/erase`	settings.manage	Erase Customer
POST	`/api/admin/docs/regenerate`	settings.manage	Docs Regenerate
GET	`/api/admin/docs/technical.md`	settings.manage	Docs Md
GET	`/api/admin/docs/technical.pdf`	settings.manage	Docs Pdf
POST	`/api/admin/recompute-counters`	settings.manage	Recompute Counters
POST	`/api/admin/reconcile-ledgers`	settings.manage	Reconcile
POST	`/api/admin/reseed`	settings.manage	Reseed

audit

Method	Path	Permission	Purpose
GET	`/api/audit-logs`		List Audit Logs
GET	`/api/audit-logs/entity-types`	audit.view	Entity Types
GET	`/api/audit-logs/entity/{entity_type}/{entity_id}`	project.read	Entity History
GET	`/api/audit-logs/login-events`	audit.view	Login Events

auth

Method	Path	Permission	Purpose
POST	`/api/auth/change-password`		Change Password
POST	`/api/auth/login`		Login
POST	`/api/auth/login/mfa`		Login Mfa
POST	`/api/auth/logout`		Logout
GET	`/api/auth/me`		Me
POST	`/api/auth/mfa/disable`		Mfa Disable
POST	`/api/auth/mfa/enroll`		Mfa Enroll
POST	`/api/auth/mfa/verify`		Mfa Verify
POST	`/api/auth/refresh`		Refresh
DELETE	`/api/auth/sessions`		Revoke All Sessions
GET	`/api/auth/sessions`		Sessions
DELETE	`/api/auth/sessions/{session_id}`		Revoke Session

blocks

Method	Path	Permission	Purpose
GET	/api/blocks		List Blocks
POST	/api/blocks		Create Block
GET	/api/blocks/{block_id}	project.read	Get Block
PATCH	/api/blocks/{block_id}	block.write	Update Block
POST	/api/blocks/{block_id}/bulk-price	flat.write	Bulk Price
POST	/api/blocks/{block_id}/bulk-status	flat.status	Bulk Status
POST	/api/blocks/{block_id}/floors	block.write	Add Floors
POST	/api/blocks/{block_id}/generate-flats	block.write	Regenerate Flats
GET	/api/blocks/{block_id}/grid	project.read	Block Grid

bookings

Method	Path	Permission	Purpose
GET	/api/bookings		List Bookings
POST	/api/bookings		Create Booking
GET	/api/bookings/{booking_id}	booking.read	Get Booking
POST	/api/bookings/{booking_id}/agreement-revisions	agreement.edit	Revise Agreement
POST	/api/bookings/{booking_id}/cancel	booking.cancel	Cancel Booking
POST	/api/bookings/{booking_id}/complete	booking.create	Complete Booking
GET	/api/bookings/{booking_id}/demands/{demand_id}/letter	booking.read	Demand Letter
GET	/api/bookings/{booking_id}/ledger	booking.read	Ledger
POST	/api/bookings/{booking_id}/other-charges/demand	booking.create	Demand Other Charge
POST	/api/bookings/{booking_id}/recompute	settings.manage	Recompute

bootstrap

Method	Path	Permission	Purpose
POST	/api/admin/recompute-read-models	settings.manage	Recompute
GET	/api/blocks/{block_id}/summary	payment.read	Block Summary
GET	/api/bootstrap		Bootstrap
GET	/api/customers/{customer_id}/summary	payment.read	Customer Summary
GET	/api/dashboard/snapshot	project.read	Dashboard Snapshot
GET	/api/projects/{project_id}/summary	payment.read	Project Summary

catalogues

Method	Path	Permission	Purpose
GET	/api/catalogues		Get Bundle
GET	/api/catalogues/{ctype}		Get Type
GET	/api/catalogues/{ctype}/export.csv		Export Csv
POST	/api/catalogues/{ctype}/import	catalogue.write	Import Csv
POST	/api/catalogues/{ctype}/items	catalogue.write	Add Item
PATCH	/api/catalogues/{ctype}/items/{item_id}	catalogue.write	Patch Item

Method	Path	Permission	Purpose
POST	/api/catalogues/{ctype}/reorder	catalogue.write	Reorder

cms

Method	Path	Permission	Purpose
POST	/api/cms/media	cms.write	Upload Media
GET	/api/cms/projects/{project_id}/website	project.read	Project Website
PATCH	/api/cms/projects/{project_id}/website	project.write	Update Project Website
GET	/api/cms/site	cms.read	Site
PATCH	/api/cms/site	cms.write	Update Site
GET	/api/cms/{coll}	cms.read	List Items
POST	/api/cms/{coll}	cms.write	Create Item
DELETE	/api/cms/{coll}/{item_id}	cms.write	Delete Item
PATCH	/api/cms/{coll}/{item_id}	cms.write	Update Item

construction

Method	Path	Permission	Purpose
POST	/api/admin/integrity-repair	settings.manage	Integrity Repair Route
GET	/api/admin/integrity-report	settings.manage	Integrity Report Route
GET	/api/admin/migration-report	settings.manage	Migration Report
POST	/api/admin/run-dues	settings.manage	Run Dues
GET	/api/blocks/{block_id}/construction	project.read	Block Construction
POST	/api/blocks/{block_id}/construction/stage	construction.update	Set Current Stage
POST	/api/blocks/{block_id}/construction/undo	construction.undo	Undo Stage
PATCH	/api/blocks/{block_id}/stage-sequence	project.write	Set Block Sequence
POST	/api/bookings/{booking_id}/payment-plan	agreement.edit	Change Booking Plan
GET	/api/bookings/{booking_id}/schedule	booking.read	Booking Schedule
GET	/api/payment-plan-assignments	project.read	List Assignments
POST	/api/payment-plan-assignments	master.write	Assign Plan
DELETE	/api/payment-plan-assignments/{assignment_id}	master.write	Unassign Plan
GET	/api/projects/{project_id}/blocks/{block_id}/payment-plans	project.read	Block Payment Plans
PATCH	/api/projects/{project_id}/stage-sequence	project.write	Set Project Sequence
GET	/api/reports/upcoming-dues	report.view	Upcoming Dues

crm

Method	Path	Permission	Purpose
GET	/api/bookings/{booking_id}/ledgers	booking.read	Booking Three Ledgers
GET	/api/communications	customer.read	List Communications
POST	/api/communications	customer.write	Log Communication
GET	/api/customers/{customer_id}/ledgers	booking.read	Customer Three Ledgers
PATCH	/api/customers/{customer_id}/profile	customer.write	Update Profile
GET	/api/customers/{customer_id}/timeline	customer.read	Customer Timeline

customers

Method	Path	Permission	Purpose
GET	/api/customers		List Customers
POST	/api/customers		Create Customer
GET	/api/customers/rm-options	customer.read	Rm Options
GET	/api/customers/{customer_id}	customer.read	Get Customer
PATCH	/api/customers/{customer_id}	customer.write	Update Customer
POST	/api/customers/{customer_id}/kyc-status	customer.kyc_view	Set Kyc Status
GET	/api/customers/{customer_id}/payments	payment.read	Customer Payments

dashboard

Method	Path	Permission	Purpose
GET	/api/dashboard		Dashboard
GET	/api/dashboard/widgets	project.read	Widgets

documents

Method	Path	Permission	Purpose
GET	/api/documents		List Documents
POST	/api/documents		Upload
DELETE	/api/documents/{doc_id}		Soft Delete
GET	/api/documents/{doc_id}/download		Download

flats

Method	Path	Permission	Purpose
GET	/api/flats		List Flats
GET	/api/flats/{flat_id}	project.read	Get Flat
PATCH	/api/flats/{flat_id}	flat.write	Update Flat
PATCH	/api/flats/{flat_id}/status	flat.status	Change Status

home

Method	Path	Permission	Purpose
GET	/api/dashboard/home	project.read	Home

imports

Method	Path	Permission	Purpose
GET	/api/imports/template/{kind}	master.read	Template
POST	/api/imports/{kind}/confirm	master.write	Confirm
POST	/api/imports/{kind}/preview	master.write	Preview

leads

Method	Path	Permission	Purpose
GET	/api/leads		List Leads
POST	/api/leads		Create Lead
GET	/api/leads/board	lead.read	Board
GET	/api/leads/stages	lead.read	Stages
GET	/api/leads/{lead_id}	lead.read	Get Lead
PATCH	/api/leads/{lead_id}	lead.write	Update Lead
POST	/api/leads/{lead_id}/convert	lead.write	Convert
POST	/api/leads/{lead_id}/follow-ups	lead.write	Add Follow Up
POST	/api/leads/{lead_id}/stage	lead.write	Move Stage

master

Method	Path	Permission	Purpose
GET	/api/master/amenities		List Amenities
POST	/api/master/amenities		Create Amenities
GET	/api/master/amenities/{item_id}		Get Amenities
PATCH	/api/master/amenities/{item_id}		Update Amenities
POST	/api/master/amenities/{item_id}/activate		Activate Amenities
POST	/api/master/amenities/{item_id}/deactivate		Deactivate Amenities
POST	/api/master/amenities/{item_id}/restore		Restore Amenities
GET	/api/master/area-types		List Area Types
POST	/api/master/area-types		Create Area Types
GET	/api/master/area-types/{item_id}		Get Area Types
PATCH	/api/master/area-types/{item_id}		Update Area Types
POST	/api/master/area-types/{item_id}/activate		Activate Area Types
POST	/api/master/area-types/{item_id}/deactivate		Deactivate Area Types
POST	/api/master/area-types/{item_id}/restore		Restore Area Types
GET	/api/master/bank-accounts		List Bank Accounts
POST	/api/master/bank-accounts		Create Bank Accounts
GET	/api/master/bank-accounts/{item_id}		Get Bank Accounts
PATCH	/api/master/bank-accounts/{item_id}		Update Bank Accounts
POST	/api/master/bank-accounts/{item_id}/activate		Activate Bank Accounts
POST	/api/master/bank-accounts/{item_id}/deactivate		Deactivate Bank Accounts
POST	/api/master/bank-accounts/{item_id}/restore		Restore Bank Accounts

Method	Path	Permission	Purpose
GET	/api/master/block-types		List Block Types
POST	/api/master/block-types		Create Block Types
GET	/api/master/block-types/{item_id}		Get Block Types
PATCH	/api/master/block-types/{item_id}		Update Block Types
POST	/api/master/block-types/{item_id}/activate		Activate Block Types
POST	/api/master/block-types/{item_id}/deactivate		Deactivate Block Types
POST	/api/master/block-types/{item_id}/restore		Restore Block Types
GET	/api/master/charge-types		List Charge Types
POST	/api/master/charge-types		Create Charge Types
GET	/api/master/charge-types/{item_id}		Get Charge Types
PATCH	/api/master/charge-types/{item_id}		Update Charge Types
POST	/api/master/charge-types/{item_id}/activate		Activate Charge Types
POST	/api/master/charge-types/{item_id}/deactivate		Deactivate Charge Types
POST	/api/master/charge-types/{item_id}/restore		Restore Charge Types
GET	/api/master/construction-stages		List Construction Stages
POST	/api/master/construction-stages		Create Construction Stages
GET	/api/master/construction-stages/{item_id}		Get Construction Stages
PATCH	/api/master/construction-stages/{item_id}		Update Construction Stages
POST	/api/master/construction-stages/{item_id}/activate		Activate Construction Stages
POST	/api/master/construction-stages/{item_id}/deactivate		Deactivate Construction Stages
POST	/api/master/construction-stages/{item_id}/restore		Restore Construction Stages
GET	/api/master/document-types		List Document Types
POST	/api/master/document-types		Create Document Types
GET	/api/master/document-types/{item_id}		Get Document Types
PATCH	/api/master/document-types/{item_id}		Update Document Types
POST	/api/master/document-types/{item_id}/activate		Activate Document Types
POST	/api/master/document-types/{item_id}/deactivate		Deactivate Document Types
POST	/api/master/document-types/{item_id}/restore		Restore Document Types
GET	/api/master/flat-types		List Flat Types
POST	/api/master/flat-types		Create Flat Types
GET	/api/master/flat-types/{item_id}		Get Flat Types
PATCH	/api/master/flat-types/{item_id}		Update Flat Types
POST	/api/master/flat-types/{item_id}/activate		Activate Flat Types
POST	/api/master/flat-types/{item_id}/deactivate		Deactivate Flat Types
POST	/api/master/flat-types/{item_id}/restore		Restore Flat Types
GET	/api/master/floor-plan-templates		List Floor Plan Templates
POST	/api/master/floor-plan-templates		Create Floor Plan Templates
GET	/api/master/floor-plan-templates/{item_id}		Get Floor Plan Templates

Method	Path	Permission	Purpose
PATCH	/api/master/floor-plan-templates/{item_id}		Update Floor Plan Templates
POST	/api/master/floor-plan-templates/{item_id}/activate		Activate Floor Plan Templates
POST	/api/master/floor-plan-templates/{item_id}/deactivate		Deactivate Floor Plan Templates
POST	/api/master/floor-plan-templates/{item_id}/restore		Restore Floor Plan Templates
GET	/api/master/payment-modes		List Payment Modes
POST	/api/master/payment-modes		Create Payment Modes
GET	/api/master/payment-modes/{item_id}		Get Payment Modes
PATCH	/api/master/payment-modes/{item_id}		Update Payment Modes
POST	/api/master/payment-modes/{item_id}/activate		Activate Payment Modes
POST	/api/master/payment-modes/{item_id}/deactivate		Deactivate Payment Modes
POST	/api/master/payment-modes/{item_id}/restore		Restore Payment Modes
GET	/api/master/payment-plans		List Payment Plans
POST	/api/master/payment-plans		Create Payment Plans
GET	/api/master/payment-plans/{item_id}		Get Payment Plans
PATCH	/api/master/payment-plans/{item_id}		Update Payment Plans
POST	/api/master/payment-plans/{item_id}/activate		Activate Payment Plans
POST	/api/master/payment-plans/{item_id}/deactivate		Deactivate Payment Plans
POST	/api/master/payment-plans/{item_id}/restore		Restore Payment Plans
GET	/api/master/plc-types		List Plc Types
POST	/api/master/plc-types		Create Plc Types
GET	/api/master/plc-types/{item_id}		Get Plc Types
PATCH	/api/master/plc-types/{item_id}		Update Plc Types
POST	/api/master/plc-types/{item_id}/activate		Activate Plc Types
POST	/api/master/plc-types/{item_id}/deactivate		Deactivate Plc Types
POST	/api/master/plc-types/{item_id}/restore		Restore Plc Types
GET	/api/master/room-types		List Room Types
POST	/api/master/room-types		Create Room Types
GET	/api/master/room-types/{item_id}		Get Room Types
PATCH	/api/master/room-types/{item_id}		Update Room Types
POST	/api/master/room-types/{item_id}/activate		Activate Room Types
POST	/api/master/room-types/{item_id}/deactivate		Deactivate Room Types
POST	/api/master/room-types/{item_id}/restore		Restore Room Types
GET	/api/master/spec-categories		List Spec Categories
POST	/api/master/spec-categories		Create Spec Categories
GET	/api/master/spec-categories/{item_id}		Get Spec Categories
PATCH	/api/master/spec-categories/{item_id}		Update Spec Categories
POST	/api/master/spec-categories/{item_id}/activate		Activate Spec Categories

Method	Path	Permission	Purpose
POST	/api/master/spec-categories/{item_id}/deactivate		Deactivate Spec Categories
POST	/api/master/spec-categories/{item_id}/restore		Restore Spec Categories
GET	/api/master/spec-items		List Spec Items
POST	/api/master/spec-items		Create Spec Items
GET	/api/master/spec-items/{item_id}		Get Spec Items
PATCH	/api/master/spec-items/{item_id}		Update Spec Items
POST	/api/master/spec-items/{item_id}/activate		Activate Spec Items
POST	/api/master/spec-items/{item_id}/deactivate		Deactivate Spec Items
POST	/api/master/spec-items/{item_id}/restore		Restore Spec Items

milestones (legacy alias)

Method	Path	Permission	Purpose
GET	/api/blocks/{block_id}/milestones	project.read	List Milestones
POST	/api/blocks/{block_id}/milestones/{seq}/complete	construction.update	Complete Milestone
POST	/api/blocks/{block_id}/milestones/{seq}/undo	construction.undo	Undo Milestone

notifications

Method	Path	Permission	Purpose
GET	/api/notifications		List Notifications
POST	/api/notifications/read-all	project.read	Mark All Read
GET	/api/notifications/unread-count	project.read	Unread Count
POST	/api/notifications/{notification_id}/read	project.read	Mark Read

parking

Method	Path	Permission	Purpose
GET	/api/parking		List Slots
POST	/api/parking		Create Slot
POST	/api/parking/bulk-generate	block.write	Bulk Generate
PATCH	/api/parking/{slot_id}	block.write	Update Slot

payments

Method	Path	Permission	Purpose
GET	/api/payments		List Payments
POST	/api/payments		Post Payment
GET	/api/payments/cheques-in-hand	payment.read	Cheques In Hand
GET	/api/payments/demands	payment.read	List Open Demands
GET	/api/payments/summary	payment.read	Payments Summary
GET	/api/payments/{payment_id}	payment.read	Get Payment
POST	/api/payments/{payment_id}/bounce	payment.post	Bounce Payment

Method	Path	Permission	Purpose
POST	/api/payments/{payment_id}/clear	payment.post	Clear Payment
GET	/api/payments/{payment_id}/receipt	receipt.print	Receipt
POST	/api/payments/{payment_id}/reverse	payment.reverse	Reverse Payment

pdf

Method	Path	Permission	Purpose
GET	/api/bookings/{booking_id}/demands/{demand_id}/letter.pdf	booking.read	Demand Letter Pdf
GET	/api/payments/{payment_id}/receipt.pdf	receipt.print	Receipt Pdf

portal

Method	Path	Permission	Purpose
POST	/api/portal/auth/change-password		Change Password
POST	/api/portal/auth/login		Login
POST	/api/portal/auth/logout		Logout
POST	/api/portal/auth/logout-all		Logout All
GET	/api/portal/auth/me		Me
POST	/api/portal/auth/refresh		Refresh
GET	/api/portal/bank-details		Bank Details
GET	/api/portal/bookings		My Bookings
GET	/api/portal/bookings/{booking_id}	booking.read	Booking Detail
GET	/api/portal/bookings/{booking_id}/allotment.pdf		Allotment Pdf
GET	/api/portal/bookings/{booking_id}/construction		Booking Construction
GET	/api/portal/bookings/{booking_id}/demands	payment.read	Booking Demands
GET	/api/portal/bookings/{booking_id}/demands/{demand_id}/letter.pdf	booking.read	Demand Pdf
GET	/api/portal/bookings/{booking_id}/documents		Booking Documents
GET	/api/portal/bookings/{booking_id}/documents/{doc_id}/download		Booking Document Download
GET	/api/portal/bookings/{booking_id}/ledger	booking.read	Booking Ledger
GET	/api/portal/bookings/{booking_id}/payments		Booking Payments
GET	/api/portal/bookings/{booking_id}/payments/{payment_id}/receipt.pdf	receipt.print	Receipt Pdf
GET	/api/portal/bookings/{booking_id}/schedule	booking.read	Booking Schedule
GET	/api/portal/bookings/{booking_id}/statement.pdf		Statement Pdf
POST	/api/portal/change-password		Change Password
GET	/api/portal/config		Portal Config
GET	/api/portal/me		Me

Method	Path	Permission	Purpose
PATCH	/api/portal/me		Update Me
GET	/api/portal/notices	cms.read	Notices
POST	/api/portal/notices/{notice_id}/read		Read Notice
GET	/api/portal/notifications		Portal Notifications
POST	/api/portal/notifications/read-all	project.read	Portal Read All
GET	/api/portal/notifications/unread-count	project.read	Portal Unread
POST	/api/portal/notifications/{notification_id}/read	project.read	Portal Read One
GET	/api/portal/service-categories		Sr Categories
GET	/api/portal/service-requests	service.read	My Requests
POST	/api/portal/service-requests	service.write	Create Request
GET	/api/portal/service-requests/{sr_id}	service.read	My Request
GET	/api/portal/service-requests/{sr_id}/attachments/{idx}	service.read	My Request Attachment
POST	/api/portal/service-requests/{sr_id}/rate		Rate Request
POST	/api/portal/service-requests/{sr_id}/reopen		Reopen Request
POST	/api/portal/service-requests/{sr_id}/reply	service.write	Reply Request

portal-admin

Method	Path	Permission	Purpose
POST	/api/customers/portal/bulk-enable		Portal Bulk Enable
GET	/api/customers/{customer_id}/portal		Portal Status
POST	/api/customers/{customer_id}/portal/enable		Portal Enable
POST	/api/customers/{customer_id}/portal/revoke-sessions		Portal Revoke
POST	/api/customers/{customer_id}/portal/toggle		Portal Toggle

project-setup

Method	Path	Permission	Purpose
GET	/api/projects/{project_id}/setup	project.read	Get Setup
PUT	/api/projects/{project_id}/setup/{section}	project.write	Save Section

projects

Method	Path	Permission	Purpose
GET	/api/projects		List Projects
POST	/api/projects		Create Project
GET	/api/projects/suggest-code	project.write	Suggest Code
GET	/api/projects/{project_id}	project.read	Get Project

Method	Path	Permission	Purpose
PATCH	/api/projects/{project_id}	project.write	Update Project

public

Method	Path	Permission	Purpose
GET	/api/public/brochure/{doc_id}		Brochure
POST	/api/public/enquiries		Enquiry
GET	/api/public/flat-types		Flat Types
GET	/api/public/gallery		Gallery
GET	/api/public/home		Home
GET	/api/public/media/photo/{photo_id}		Construction Photo
GET	/api/public/media/{media_id}		Media
GET	/api/public/pages/{slug}		Page
GET	/api/public/posts		Posts
GET	/api/public/posts/{slug}		Post
GET	/api/public/projects		Projects
GET	/api/public/projects/{slug}		Project Detail
GET	/api/public/published-projects		Published Projects
GET	/api/public/robots.txt		Robots
GET	/api/public/site	cms.read	Site
GET	/api/public/sitemap.xml		Sitemap

public (visits & brochure)

Method	Path	Permission	Purpose
POST	/api/public/brochure-requests		Brochure Request
GET	/api/public/brochure/{token}		Brochure Download
POST	/api/public/site-visits		Public Book
GET	/api/public/site-visits/availability	visit.read	Public Availability
GET	/api/public/site-visits/{token}		Visit Public Detail
GET	/api/public/site-visits/{token}.ics		Visit Ics
POST	/api/public/site-visits/{token}/cancel		Visit Public Cancel

rate-cards

Method	Path	Permission	Purpose
GET	/api/projects/{project_id}/rate-cards		List Rate Cards
POST	/api/projects/{project_id}/rate-cards		Create Rate Card
GET	/api/projects/{project_id}/rate-cards/{rc_id}	project.read	Get Rate Card
PATCH	/api/projects/{project_id}/rate-cards/{rc_id}	flat.write	Update Rate Card
POST	/api/projects/{project_id}/rate-cards/{rc_id}/activate	flat.write	Activate Rate Card
POST	/api/projects/{project_id}/rate-cards/{rc_id}/apply	flat.write	Apply Rate Card

reports

Method	Path	Permission	Purpose
GET	/api/reports		List Reports
GET	/api/reports/activity	audit.view	Activity
GET	/api/reports/advances	report.view	Advances
GET	/api/reports/allotment/{booking_id}	booking.read	Allotment
GET	/api/reports/bank-receipts	report.view	Bank Receipts
GET	/api/reports/cancellations	report.view	Cancellations
GET	/api/reports/cheques	report.view	Cheques
GET	/api/reports/collections	report.view	Collections
GET	/api/reports/construction-progress	report.view	Construction Progress
GET	/api/reports/demand-vs-collection	report.view	Demand Vs Collection
GET	/api/reports/demand-vs-collection-installment		Demand Vs Collection
GET	/api/reports/inventory	report.view	Inventory
GET	/api/reports/outstanding	report.view	Outstanding
GET	/api/reports/presets	report.view	List Presets
POST	/api/reports/presets	report.view	Create Preset
DELETE	/api/reports/presets/{preset_id}	report.view	Delete Preset
PATCH	/api/reports/presets/{preset_id}	report.view	Rename Preset
GET	/api/reports/roznamcha-range	report.view	Roznamcha Range
GET	/api/reports/sales-leads	report.view	Sales Leads
GET	/api/reports/service-sla	report.view	Service Sla
GET	/api/reports/statement/{booking_id}	booking.read	Statement

roles

Method	Path	Permission	Purpose
GET	/api/permissions		List Permissions
GET	/api/roles		List Roles
POST	/api/roles	roles.manage	Create Role
DELETE	/api/roles/{role_id}	roles.manage	Delete Role
GET	/api/roles/{role_id}	roles.manage	Get Role
PATCH	/api/roles/{role_id}	roles.manage	Update Role

roznamcha

Method	Path	Permission	Purpose
GET	/api/roznamcha		Day Book
POST	/api/roznamcha/contra	payment.post	Contra
GET	/api/roznamcha/range	payment.read	Day Book Range

search

Method	Path	Permission	Purpose
GET	/api/search		Global Search

settings

Method	Path	Permission	Purpose
GET	/api/settings/company		Get Company
PATCH	/api/settings/company	settings.manage	Update Company
GET	/api/settings/numbering/preview	settings.manage	Numbering Preview

site visits

Method	Path	Permission	Purpose
GET	/api/site-visits		List Visits
POST	/api/site-visits		Erp Create
GET	/api/site-visits/availability	visit.read	Erp Availability
GET	/api/site-visits/brochure-stats	lead.read	Brochure Stats
GET	/api/site-visits/calendar	visit.read	Calendar
GET	/api/site-visits/export	visit.read	Export Visits
GET	/api/site-visits/today	visit.read	Today Visits
PATCH	/api/site-visits/{visit_id}	visit.write	Visit Notes
POST	/api/site-visits/{visit_id}/reschedule	visit.write	Reschedule
POST	/api/site-visits/{visit_id}/{action}	visit.write	Visit Action

support

Method	Path	Permission	Purpose
PATCH	/api/blocks/{block_id}/milestones/{seq}/note	milestone.update	Milestone Note
POST	/api/blocks/{block_id}/milestones/{seq}/photos	milestone.update	Upload Photos
GET	/api/blocks/{block_id}/photos	project.read	List Photos
PATCH	/api/documents/{doc_id}/share	customer.write	Share Document
GET	/api/notices	cms.read	List Notices
POST	/api/notices	cms.write	Create Notice
DELETE	/api/notices/{notice_id}	cms.write	Delete Notice
PATCH	/api/notices/{notice_id}	cms.write	Update Notice
DELETE	/api/photos/{photo_id}	milestone.update	Delete Photo
PATCH	/api/photos/{photo_id}	milestone.update	Patch Photo
GET	/api/service-requests	service.read	List Sr
POST	/api/service-requests	service.write	Create Sr
GET	/api/service-requests/categories	service.read	Sr Categories
GET	/api/service-requests/{sr_id}	service.read	Get Sr
POST	/api/service-requests/{sr_id}/assign	service.assign	Assign Sr
GET	/api/service-requests/{sr_id}/attachments/{idx}	service.read	Sr Attachment
POST	/api/service-requests/{sr_id}/reply	service.write	Reply Sr
POST	/api/service-requests/{sr_id}/status	service.write	Status Sr

system

Method	Path	Permission	Purpose
GET	/api/health		Health
GET	/api/metrics		Metrics
GET	/api/ready		Ready

users

Method	Path	Permission	Purpose
GET	/api/users		List Users
POST	/api/users		Create User
GET	/api/users/{user_id}	users.manage	Get User
PATCH	/api/users/{user_id}	users.manage	Update User
POST	/api/users/{user_id}/reset-password	users.manage	Reset Password

B. Error code catalogue (generated)

Every error is {detail, code, request_id}; HTTP status is the first match in code. VERSION_CONFLICT (409) is returned when version in the body is stale (optimistic concurrency).

Code	HTTP	Message (example)	Raised in
ACCOUNT_LOCKED	423	Account locked due to repeated failed attempts. Try again after {LOCK_MINUTES} minutes.	routers/auth.py
ACTIVE_BOOKING	409	Customer has an active or completed booking; erasure not allowed	routers/admin.py
ALREADY_COMPLETED	409	Stage already completed	routers/construction.py
ALREADY_DEMANDED	409	Charge already demanded	routers/bookings.py
AREA_HIERARCHY	422	Carpet area ({carpet}) cannot exceed built-up area ({builtup})	routers/master.py
BELOW_PAID	409	Cash agreed cannot be below cash already received ({present_money({'x_paise': s.get('cash_	routers/bookings.py
BOOKING_NOT_ACTIVE	409	Booking is {booking['status']}; payments are not accepted	routers/payments.py
BUCKET_EXCEEDS_AGREED	409	{category} bucket agreed {inr(agreed)}; already received {inr(paid)}; cannot accept {inr(a	routers/payments.py
CANNOT_REOPEN	409	Cannot reopen (only within 7 days of resolution)	routers/portal.py
CHARGE_EXCEEDS_TOTAL	409	{line['name']}: total {inr(line['total_paise'])}; already received {inr(paid)}; cannot acc	routers/payments.py

Code	HTTP	Message (example)	Raised in
CLOSED	409	Ticket is closed	routers/portal.py
DOCS_BUILD_FAILED	500	Docs build failed: {out.decode()[-400:]}	routers/admin.py
DUPLICATE	409	Slot {body.number} already exists	routers/parking.py
DUPLICATE_CODE	409	Block code {body.code} already exists in this project	routers/blocks.py
DUPLICATE_EMAIL	409	A user with this email already exists	routers/users.py
DUPLICATE_NAME	409	'{body.label}' already exists in {ctype}	routers/catalogues.py
DUPLICATE_PAN	409	PAN already registered to {dup['full_name']} ({dup['customer_code']})	routers/customers.py
DUPLICATE_PHONE	409	Phone already registered to {dup['full_name']} ({dup['customer_code']})	routers/customers.py
DUPLICATE_SLUG	409	Slug '{data['slug']}' already exists	routers/cms.py
EMPTY_CSV	422	CSV has no data rows	routers/imports.py
FLATS_NOT_AVAILABLE	409	Cannot change floor plans: {n} flat(s) are not Available	routers/blocks.py
FLAT_HELD_BY_OTHER	409	Flat {flat['flat_number']} is on Hold by another user	routers/bookings.py
FLAT_NOT_AVAILABLE	409	Flat {flat['flat_number']} is {flat['status']}	routers/bookings.py
FLOOR_PLAN_GAP	422	No floor plan range covers floor {f}	routers/blocks.py
FORBIDDEN	403	Missing permission: {permission}	deps.py
GATE_DISABLED	409	Brochure gate is disabled for this project	routers/visits.py
IDEMPOTENCY_KEY_REQUIRED	400	Idempotency-Key header is required when posting payments	routers/payments.py
INSUFFICIENT_CASH	409	Cash in hand on {body.date} is only ₹{available / 100:,.2f}	routers/roznamcha.py

Code	HTTP	Message (example)	Raised in
INVALID_AMENITY	400	One or more amenities are invalid or inactive	routers/projects.py
INVALID_AREA_TYPE	400	One or more area types are invalid or inactive	routers/master.py
INVALID_AUDIENCE	401	Token not valid for ERP	deps.py
INVALID_BANK_ACCOUNT	400	Escrow bank account not found	routers/projects.py
INVALID_BLOCK_TYPE	400	Block type not found	routers/blocks.py
INVALID_CHARGE_TYPE	400	Unknown charge type	routers/bookings.py
INVALID_CO_APPLICANT	400	Unknown co-applicant customer	routers/bookings.py
INVALID_CREDENTIALS	401	Invalid email or password	routers/auth.py
INVALID_FLAT_TYPE	400	One or more flat types are invalid	routers/rate_cards.py
INVALID_FLOOR_PLAN	400	One or more floor plan templates are invalid or inactive	services/flat_generation.py
INVALID_MILESTONE_TEMPLATE	400	Milestone template not found	routers/blocks.py
INVALID_PLC_TYPE	400	One or more PLC types are invalid	routers/rate_cards.py
INVALID_POSITIONS	422	position_flat_types must cover positions 1..N exactly once	routers/blocks.py
INVALID_REFRESH_TOKEN	401	Invalid refresh token	routers/auth.py
INVALID_ROLE	400	Role not found	routers/users.py
INVALID_ROOM_TYPE	400	One or more room types are invalid or inactive	routers/master.py
INVALID_SCOPE	400	One or more projects in scope do not exist	routers/users.py
INVALID_SPEC_ITEM	400	One or more specification items are invalid or inactive	routers/master.py

Code	HTTP	Message (example)	Raised in
INVALID_STATE	409	Visit is already {v['status']}	routers/visits.py
INVALID_TEAM	400	One or more team members are invalid	routers/projects.py
INVALID_TOKEN	401	Invalid token	deps.py
IN_USE	409	Cannot deactivate: block has {n} flat(s) not Available	routers/blocks.py
LAST_SUPER_ADMIN	409	Cannot remove the last active Super Admin	routers/users.py
LEAD_CONVERTED	409	Converted leads cannot be moved	routers/leads.py
LINK_EXPIRED	410	This link has expired	routers/visits.py
LINK_REVOKED	410	This link is no longer valid	routers/visits.py
LOCKED	423	Account locked. Try again later.	routers/portal.py
MFA_ENABLED	409	MFA is already enabled	routers/auth.py
MFA_INVALID	401	Invalid authentication code	routers/auth.py
NOTHING_TO_UNDO	409	No completed stage to undo	routers/construction.py
NOT_ACTIVE	409	Activate the rate card before applying	routers/rate_cards.py
NOT_AVAILABLE	409	Customer import becomes available in Phase 2	routers/imports.py
NOT_CLEARED	409	Only cleared payments can be reversed (status {p['status']})	routers/payments.py
NOT_DRAFT	409	Only Draft rate cards can be edited; create a new card to supersede	routers/rate_cards.py
NOT_FOUND	404	{entity} not found	utils.py
NOT_PENDING	409	Payment is {p['status']}	routers/payments.py

Code	HTTP	Message (example)	Raised in
NO_BROCHURE	404	This project has no brochure yet	routers/visits.py
NO_CHANGE	409	Flat is already {body.status}	routers/flats.py
NO_CHANGES	400	No changes supplied	routers/blocks.py
PARKING_NOT_AVAILABLE	409	One or more parking slots are not available	routers/bookings.py
PASSWORD_CHANGE_REQUIRED	403	Password change required	routers/portal.py
PASSWORD_REUSED	422	Password was used recently; choose one not among your last {PASSWORD_HISTORY}	routers/auth.py
PAYLOAD_TOO_LARGE	413	File must be 1 byte – 15 MB	routers/cms.py
PENDING_PAYMENTS	409	Clear or bounce pending cheques before cancelling	routers/bookings.py
PLAN_LOCKED	409	An installment demand already has a payment; plan cannot be changed	routers/construction.py
PLAN_NOT_APPLICABLE	409	Plan is not assigned to this tower	routers/construction.py
PLAN_STAGES_MISSING	409	Plan '{plan['name']}' needs stages missing from {block['name']}'s sequence: {', '.join(nam	routers/construction.py
RATE_LIMITED	429	Rate limit exceeded ({limit}/{window_s}s). Retry in {retry}s.	deps.py
REFRESH_TOKEN_EXPIRED	401	Refresh token expired	routers/auth.py
REFRESH_TOKEN_REUSED	401	Refresh token reuse detected; all sessions revoked	routers/auth.py
REFUND_EXCEEDS_PAID	409	Refund cannot exceed amount received	routers/bookings.py
SELF_DEACTIVATE	409	You cannot deactivate your own account	routers/users.py
SENSITIVE_DOCUMENT	409	Sensitive KYC documents cannot be shared with the portal	routers/support.py
SEQUENCE_LOCKED	409	Cannot change the sequence once stages are completed; undo them first	routers/construction.py

Code	HTTP	Message (example)	Raised in
SLOT_FULL	409	This slot is now full — please pick another	routers/visits.py
SLOT_UNAVAILABLE	409	That slot is not available	routers/visits.py
STAGE_COMPLETED_LOCKED	409	Cannot remove a completed stage from a tower with active bookings	routers/project_setup.py
STATUS_LOCKED	409	Allotted slots are controlled by the booking	routers/parking.py
STATUS_NOT_MANUAL	409	Status {body.status} is set by the booking engine and cannot be set manually	routers/flats.py
STORAGE_ERROR	502	Object storage upload failed: {e}	routers/cms.py
SYSTEM_ROLE	409	System roles cannot be renamed or deactivated	routers/roles.py
TOKEN_EXPIRED	401	Access token expired	deps.py
TOKEN_REUSE	401	Refresh token reuse detected; all sessions revoked	routers/portal.py
TOO_MANY_ROWS	422	Max 5000 rows per import	routers/imports.py
TOWER_NOT_IN_PROJECT	404	Tower does not belong to this project	routers/project_setup.py
UNAUTHENTICATED	401	Not authenticated	deps.py
UNDO_BLOCKED_BY_PAYMENTS	409	A demand raised by this stage already has a payment; undo is blocked	routers/construction.py
UNKNOWN_PARAM	400	Unknown query parameter(s): {', '.join(unknown)}. Accepted: {', '.join(sorted(allowed))}	routers/reports.py
UNSUPPORTED_MEDIA	422	Only JPG/PNG/WEBP/SVG/ICO/PDF allowed	routers/cms.py
USER_INACTIVE	401	User inactive or not found	deps.py
USE_CONVERT	409	Use /convert to convert a lead	routers/leads.py
VALIDATION_ERROR	422	Provide rate_per_sqft or price	routers/blocks.py

Code	HTTP	Message (example)	Raised in
VERSION_CONFLICT	409	{entity} was modified by someone else (current version {current.get('version') if current	utils.py
VISITS_DISABLED	409	Site visits are not open for this project	routers/visits.py

C. Settings keys

Key	Collection	Meaning
roles_version	settings	bumps on role edits; invalidates permission caches
seed_done, seed_phase2_done … seed_phase9_done	settings	seed guards
round_robin_assignment	site_settings	lead assignment mode
demand_due_days	companies	default days from demand to due date (15)
receipt_prefix, *_number_format	companies	numbering patterns (BK-{FY}-{SEQ}, {PREFIX}/{FY}/{SEQ}, CUST-…, DM-…, LD-…, SV-…)
website.site_visits.*, website.brochure_gate_enabled	projects	per-project site behaviour

D. Status & enum tables (generated)

Constant	File	Values
STAGES	routers/leads.py	New, Contacted, Site Visit Scheduled, Site Visit Done, Negotiation, Converted, Lost
SR_STATUSES	routers/support.py	New, In Progress, Awaiting Customer, Resolved, Closed
VISIT_STATUSES	routers/visits.py	Confirmed, Rescheduled, Completed, No-show, Cancelled

E. Seeded demo data & credentials

Item	Value
Company	Shree Ganesh Builders, Patna (FY Apr–Mar)
Projects	Shree Ganesh Heights (SGH: Towers A 45 %, B 18 %, C 0 %), Ganesh Green Villas (GGV: Villa Blocks P 100 %, V 67 %)
ERP users	admin@kkt.in (Super Admin), projectadmin@kkt.in, sales@kkt.in, accounts@kkt.in, marketing@kkt.in, engineer@kkt.in, employee@kkt.in — password Kkt@12345
Portal	12 customers with portal accounts (see /app/memory/test_credentials.md)
Bookings	BK-2526-0001 … 0008 with full ledgers; Tower B bookings carry stage-linked demands
Site visits	SV-2627-0001 … 0010 (6 upcoming, 4 past)
Brochures	generated PDFs for both projects; gate enabled

F. Glossary

Term	Meaning
CLP	Construction-linked plan — installments fall due as construction stages complete
PLC	Preferential location charge — premium for floor, facing, corner, view
RERA	Real Estate (Regulation and Development) Act registration; number shown on every project
Super built-up area	Carpet + walls + proportionate common areas; the area pricing is quoted on
Carpet area	Usable floor area within walls (RERA definition)
Roznamcha	Daily cash book — opening, receipts, payments, closing cash
Demand (letter)	Formal request for an installment with due date and bank details
Agreement value / buckets	Total consideration split into Account (banked) and Cash buckets per the agreement
Allotment letter	Document confirming the unit allotted to the buyer
Possession	Handover stage; final installment trigger
Refuge floor	Fire-safety floor without saleable units
Podium	Parking/amenity levels below the first residential floor
FY	Indian financial year (April–March), used in numbering 2526 = 2025–26
Idempotency-Key	Client-supplied header making money/inventory POSTs safe to retry